



**Escuela Superior
de Ingeniería y Tecnología**
Universidad de La Laguna

Trabajo de Fin de Grado

Grado en Ingeniería Informática

Análisis de datos para la gestión de residuos de las Islas Canarias

Estudio de modelos lineales de estimación

Noelia Ibáñez Silvestre

La Laguna, 9 de julio de 2025

D./Dña **José Marcos Moreno Vega**, Catedrático de Universidad adscrito al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como tutor

D./Dña **María Belén Melián Batista**, Catedrática de Universidad adscrita al Departamento de Ingeniería Informática y de Sistemas de la Universidad de La Laguna, como cotutora.

C E R T I F I C A N

Que la presente memoria titulada:

Análisis de datos para la gestión de residuos de las Islas Canarias. Estudio de modelos lineales de estimación

ha sido realizada bajo su dirección por **Noelia Ibáñez Silvestre**.

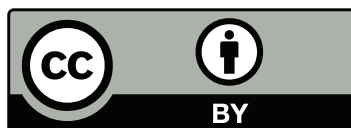
Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos, firman la presente en La Laguna a 9 de julio de 2025

Agradecimientos

A los tutores que han hecho posible este
trabajo.

A mi familia y amigos por su apoyo.

Licencia



© Esta obra está bajo una licencia de Creative Commons Reconocimiento 4.0 Internacional.

Resumen

El presente trabajo tiene como objetivo desarrollar un modelo capaz de predecir con fiabilidad la cantidad de residuos generados en la isla de Tenerife, utilizando Python como herramienta principal. Para ello, se integraron datos públicos procedentes del Instituto Canario de Estadística (ISTAC), Cabildo de Tenerife e Instituto Nacional de Estadística (INE), incluyendo variables sociodemográficas, turísticas y económicas.

Tras un proceso de limpieza y unificación de los distintos datasets, se aplicaron técnicas de regresión lineal y árboles de regresión para modelar y poder predecir la cantidad de peso vertido. Mientras que los modelos de regresión lineal muestran un margen de mejora, los árboles de regresión ofrecen resultados más precisos y satisfactorios.

Keywords: Python, predicción de residuos, modelos predictivos, regresión lineal, árboles de regresión, Tenerife, análisis de datos, turismo, población, estadísticas oficiales

Abstract

The aim of this work is to develop a reliable model capable of predicting the amount of waste generated on the island of Tenerife, using Python as the primary tool. To achieve this, public data from the Canary Islands Statistics Institute (ISTAC), the Tenerife Island Council, and the National Statistics Institute (INE) were integrated, including sociodemographic, tourism, and economic variables.

After a process of cleaning and unifying the various datasets, linear regression and regression tree techniques were applied to model and predict the amount of waste generated. While the linear regression models show room for improvement, the regression trees provide more accurate and satisfactory results.

Keywords: Python, waste prediction, predictive models, linear regression, regression trees, Tenerife, data analysis, tourism, population, official statistics

Índice general

Capítulo 1. Introducción	1
Capítulo 2. Análisis y procesamiento de los datos	2
2.1. Datos de residuos	2
2.2. Datos de población	4
2.3. Datos de turismo	6
2.4. Datos de empresas por municipios y actividad económica	8
Capítulo 3. Regresión lineal	9
3.1. Fundamentos teóricos	9
3.1.1. ¿Qué es la regresión lineal?	9
3.1.2. Regresión lineal simple vs múltiple	10
3.1.3. Hipótesis de partida	10
3.1.4. Métricas de evaluación	11
3.2. Regresión lineal simple	13
3.3. Regresión lineal múltiple	18
Capítulo 4. Árboles de regresión	21
4.1. Fundamentos teóricos	21
4.1.1. Definición	21
4.1.2. Descripción del algoritmo de aprendizaje	22
4.2. Implementación del modelo	23
4.2.1. Versión 1	24
4.2.2. Versión 2	27
4.2.3. Versión 3	29
Capítulo 5. Conclusiones	33
Capítulo 6. Conclusions	34
Capítulo 7. Presupuesto	35
Apéndice A. Algoritmos usados para el análisis y procesamiento de los datos	36
A.1. Función procesar_dataframe_residuos	36
A.2. Función ajustar_tipos_variables	37
A.3. Función modificar_municipio_nombre	38
A.4. Algoritmo df_turismo_completo	39
Apéndice B. Algoritmos usados para la regresión lineal	42
B.1. Preparación df_ar_tur	42
B.2. Evaluación de supuestos	43

B.3. Transformación a modelo logarítmico - logarítmico	44
--	----

Apéndice C. Algoritmos usados para los árboles de regresión 45

C.1. Preparación df_ar_tur	45
C.2. Preparación df_ar_pob	46
C.3. Preparación df_ar_res	47
C.4. Creación df_arbol_v1	48
C.5. Inicialización del modelo	49
C.6. Árbol de regresión v1	49
C.7. Árbol de regresión v2	51
C.8. Árbol de regresión v3	53

Índice de figuras

3.1. Resultado ejecución código 3.1	15
3.2. Resultados modelo OLS	16
3.3. Resultados OLS modelo múltiple	19
4.1. Estructura de un árbol de regresión	21
4.2. Estudio max_depth	25
4.3. Estudio min_samples_leaf	25
4.4. Estudio max_leaf_nodes	26
4.5. Árbol de regresión v1	26
4.6. df_arbol_regresion_Puerto_de_la_Cruz	28

Índice de tablas

2.1. Datos del dataframe df_residuos10	3
2.2. df_residuos_total	4
2.3. Dataframe df_poblacion_por_municipios_y_años	5
2.4. Dataframe df_turismo	6
2.5. df_turismo_completo	7
2.6. df_empresas_por_municipios_y_act_economica	8
3.1. Dataframe regresión lineal simple	13
3.2. Resultados tests de supuestos en regresión lineal múltiple	20
3.3. Resultados tests de supuestos en regresión lineal múltiple después de la transformación	20

4.1.	df_arbol_v1	24
4.2.	df_modelo_puerto_de_la_cruz	27
4.3.	Resultados del árbol de regresión por municipio	30
4.4.	Porcentajes errores por municipio	31
7.1.	Presupuesto	35

Índice de algoritmos

3.1.	Código regresión lineal simple ejemplo 1	14
3.2.	D'Agostino's K-squared test en regresión lineal simple	17
3.3.	Breusch-Pagan test en regresión lineal simple	17
3.4.	Durbin-Watson test en regresión lineal simple	18
3.5.	Código para la regresión lineal múltiple	19
4.1.	Porcentaje RMSE y MAE	28

Capítulo 1

Introducción

En la actualidad, uno de los temas de mayor relevancia para la sociedad es el cambio climático y sus consecuencias. Esta preocupación se manifiesta en numerosas iniciativas sociales y políticas. Un ejemplo reciente son las movilizaciones convocadas en junio de 2024 por la red de organizaciones climáticas *Alianza por el clima*, que reunieron a miles de personas en 113 ciudades europeas - incluyendo ocho españolas- para reclamar que el cambio climático ocupe un lugar central en los debates políticos. [1]

Otra muestra de este compromiso es la regularidad con la que se celebran encuentros internacionales, como la Cumbre Mundial del Clima, cuya 29ª edición se celebró en Bakú [2]. Estas acciones reflejan que la preocupación por el medio ambiente no es exclusiva de gobiernos o instituciones, sino también de una ciudadanía cada vez más consciente y activa.

Este compromiso colectivo también se ve reflejado en iniciativas globales como la Agenda 2030, que establece una serie de Objetivos de Desarrollo Sostenible (ODS) orientados a proteger el medio ambiente y garantizar condiciones de vida dignas para las generaciones futuras [3]. Dentro de este marco, se subraya la importancia de la actuación local como eje para alcanzar los objetivos marcados a nivel global. Entre ellos, los más directamente relacionados con la gestión de residuos son:

- ODS 11: lograr que las ciudades sean más inclusivas, seguras, resilientes y sostenibles. Además, la meta 11.6 menciona específicamente reducir el impacto ambiental negativo per cápita de ciudades, prestando especial atención a la calidad del aire y la gestión de los desechos municipales. [4]
- ODS 12: garantizar modalidades de consumo y producción sostenibles. Este objetivo, en sus metas 12.4 y 12.5 plantea la necesidad de una reducción en la generación de los residuos y una adecuada gestión de los mismos. [5]

En el contexto específico de las Islas Canarias, la gestión de residuos representa un desafío crucial dentro de esta lucha colectiva. Factores geográficos, como la insularidad y la limitada disponibilidad de espacio, junto con condicionantes socioeconómicos, dificultan una gestión eficaz y sostenible de los residuos.

Por ello, el objetivo principal de este trabajo es identificar un modelo basado en datos con el que hacer predicciones fiables de la cantidad de residuos generados en la isla de Tenerife.

Cabe señalar que, en un principio, la propuesta original del trabajo incluía el estudio de todas las islas del archipiélago. Sin embargo, debido a la falta de datos públicos sobre residuos en la mayoría de ellas, el análisis se ha limitado finalmente a la isla de Tenerife, la única con información accesible y completa.

Capítulo 2

Análisis y procesamiento de los datos

Para abordar el análisis de la gestión de residuos, resulta imprescindible comenzar por la obtención y comprensión de los datos disponibles. Esta etapa inicial constituye la base sobre la cual se construirá el resto del trabajo, ya que permite contextualizar el fenómeno estudiado y garantizar la fiabilidad del análisis posterior.

En el presente capítulo se expone el proceso de recopilación de los distintos conjuntos de datos utilizados, así como el tratamiento aplicado a los mismos con el objetivo de prepararlos adecuadamente para su uso posterior. Este tratamiento incluye tareas de limpieza, transformación y análisis exploratorio.

Todo este procesamiento de los datos se ha llevado a cabo utilizando el lenguaje de programación Python, empleando bibliotecas especializadas como *Pandas* para la gestión de datos estructurados ó *Scikit-Learn* para modelos de machine learning, entre otras. Los distintos conjuntos de datos se almacenaron y manipularon en estructuras del tipo *Dataframe*.

Para el estudio se emplearon datos sobre los residuos de Tenerife, así como información sobre población, turismo, empresas y actividad económica.

2.1. Datos de residuos

Para la elaboración del dataframe final `df_residuos_total`, se partió de diversas fuentes de datos correspondientes a los registros anuales de residuos municipales. Estos conjuntos de datos fueron obtenidos del portal de datos abiertos del Cabido de Tenerife (<https://datos.tenerife.es/es/>).

El proceso inicial consistió en la extracción de todos los dataframes disponibles desde el año 2010 hasta 2023, los cuales se nombraron siguiendo el patrón `df_residuosX`, donde X indica el año al que corresponden los datos. En la tabla 2.1 se muestra la estructura del dataframe `df_residuos10`, que contiene la información de residuos del año 2010.

municipio_nombre	planta_nombre	planta_vertedero	tipo_vertido_id	tipo_vertido_nombre	tercero_codigo_postal	tercero_ayuntamiento	tercero_particular	fecha	peso
Adeje	PT-1-La Guancha	False	200302	Residuos de mercados	NaN	False	False	NaN	0.52
Adeje	PT-1-La Guancha	False	200399	Residuos municipales no especificados en otra categoría	NaN	False	False	NaN	0.20
Adeje	PT-2-La Orotava	False	200301	Mezclas de residuos municipales	NaN	False	False	NaN	1.32
Adeje	PT-3-Arona	False	200108	Residuos biodegradables de cocinas y restaurantes	NaN	False	False	NaN	4.14

Tabla 2.1: Datos del dataframe df_residuos10

A continuación, se describen los campos presentes en este conjunto de datos, tal como están definidos en el diccionario del propio [dataset](#) [6]:

- *municipio_nombre*: nombre del municipio donde se localiza la empresa / organismo que realiza el vertido.
- *planta_nombre*: nombre de la instalación donde se realiza el vertido.
- *planta_vertedero*: indica si el vertido procede una planta de transferencia o es un vertido directo en el Complejo Ambiental. Valores: True/ False.
- *tipo_vertido_id*: código europeo de residuo (*LER*).
- *tipo_vertido_nombre*: denominación del tipo de residuo (*LER*).
- *tercero_ayuntamiento*: indica si el vertido procede del servicio de recogida municipal. Valores: True/ False.
- *tercero_particular*: indica si el vertido procede de una empresa/particular a la que se le factura mensualmente. Valores: True/ False.
- *fecha*: fecha en la que se realizó el vertido.
- *peso_vertido*: peso del vertido, en toneladas.
- *año*: año en el que se realizó el vertido.

Una vez cargados todos los dataframes individuales de residuos, y dada la estructura idéntica, se aplicó un análisis y ajuste mediante la función `procesar_dataframe_residuos`, la cual automatiza los siguientes pasos (Anexo A.1):

1. Análisis descriptivo inicial: se examina la estructura general del dataframe, se visualizan las primeras filas y se calculan estadísticas descriptivas básicas como la media, desviación estándar, etc. Además, se modifica el nombre de la columna `municipio_nombre` para que sea más legible.
2. Generación de reporte: se utiliza la biblioteca `ydata_profiling` para generar un informe interactivo detallado del dataframe.
3. Creación de la columna `año`: se extrae el año del nombre del dataframe y se crea una nueva columna con dicho valor. Esto es útil en aquellos casos en que el año no está indicado directamente en los datos y es necesario para, posteriormente, diferenciarlos en `df_residuos_total`.

4. Eliminación de la columna `tercero_codigo_postal`: se elimina la columna `tercero_codigo_postal` ya que no es relevante y no aporta información relevante en el análisis.
5. Ajuste de tipos de variables: se llama a la función definida previamente `ajustar_tipos_variables` (Anexo A.2) para corregir los tipos de datos en cada columna.
6. Separación de mancomunidades: se identifican las filas cuyo `municipio_nombre` pertenece a una mancomunidad, separándolas en otro dataframe denominado `registros_mancomunidades`. Esta decisión se toma debido a que se necesitan los datos por municipio individual, y una mancomunidad es una agrupación de municipios.
7. Modificación de nombres de municipios: se aplica la función `modificar_nombre_municipio` (Anexo A.3) a la columna `municipio_nombre` para estandarizar los nombres de los municipios.

Posteriormente, los dataframes procesados se almacenan en un diccionario llamado `dataframes_procesados`. Finalmente, todos ellos se concatenan para formar el dataframe final `df_residuos_total`, que integra la información correspondiente a todos los años disponibles.

Al finalizar el proceso, la estructura del dataframe resultante `df_residuos_total` es la siguiente (tabla 2.2):

	<code>municipio_nombre</code>	<code>planta_nombre</code>	<code>planta_vertedero</code>	<code>tipo_vertido_id</code>	<code>tipo_vertido_nombre</code>	<code>tercero_ayuntamiento</code>	<code>tercero_particular</code>	<code>fecha</code>	<code>peso</code>	<code>año</code>
<code>df_residuos10</code>	Adeje	PT-1-La Guancha	False	200302	Residuos de mercados	False	False	NaN	0.52	10
	Adeje	PT-1-La Guancha	False	200399	Residuos municipales no especificados en otra categoría	False	False	NaN	0.20	10
	Adeje	PT-2-La Orotava	False	200301	Mezclas de residuos municipales	False	False	NaN	1.32	10
	Adeje	PT-3-Arona	False	200108	Residuos biodegradables de cocinas y restaurantes	False	False	NaN	4.14	10
...	...	...	...	...	...	...	...	...	...	...
<code>df_residuos23</code>	Sin Asignar	PTMRD - Residuos voluminosos	True	200307	Residuos voluminosos	False	True	2023-12-18 15:26:00	1.12	23
	Sin Asignar	PTMRD - Residuos voluminosos	True	200307	Residuos voluminosos	False	True	2023-12-26 14:57:00	1.12	23

Tabla 2.2: `df_residuos_total`

2.2. Datos de población

El dataframe `df_poblacion_por_municipios_y_años` se obtuvo a partir del dataset [Población según sexos. Municipios por islas de Canarias y años \[7\]](#) del Instituto Canario de Estadística (ISTAC).

Tras una primera carga y la generación del informe con `ProfileReport`, se identificaron diversos aspectos a corregir. En primer lugar, se renombraron las columnas para facilitar su comprensión, y se eliminaron las columnas `'CONFIDENCIALIDAD_OBSERVACION#es'` y `'NOTAS_OBSERVACION#es'`, por su irrelevancia para el estudio.

El informe también relevó la presencia de registros de otras islas, años fuera del periodo de interés (2010-2023) y algunas filas vacías. Para solucionar esto, se aplicó un filtrado que permitió conservar únicamente los datos correspondientes a los municipios de Tenerife dentro del rango temporal establecido, lo que también implicó la eliminación automática de las filas vacías.

Finalmente, se seleccionaron las columnas necesarias y se ajustaron los tipos de datos para garantizar la correcta manipulación de estos. El resultado fue un dataframe estructurado (tabla 2.3) con las siguientes variables:

- *tipo_medida*: indicador demográfico registrado. Incluye:
 - ◊ *Población*: número de personas según las cifras oficiales de población a 1 de enero de cada año, elaboradas a partir del Padrón Municipal de Habitantes.
 - ◊ *Población. Variación anual*: diferencia absoluta entre la población del año de referencia y el anterior.
 - ◊ *Población. Tasa de variación anual*: tasa de cambio porcentual anual, calculada como el cociente entre la variación absoluta y la población del año anterior.
 - ◊ *Población. Proporción*: proporción que representa un subconjunto poblacional dentro del total.
- *año*: año al que se refiere la observación.
- *territorio*: municipio de Tenerife al que corresponde el registro.
- *sexo*: sexo de la población registrada (mujeres, hombres o total).
- *valor*: cantidad de personas registradas.

En la tabla 2.3 se muestran algunas de las filas del dataframe resultante llamado `df_poblacion_por_municipios_y_años`.

tipo_medida	año	territorio	sexo	valor
Población	2010	Tenerife	Mujeres	457527.00
Población. Proporción	2010	Tenerife	Mujeres	43.17
Población. Tasa de variación anual	2010	Tenerife	Mujeres	0.85
Población. Variación anual	2010	Tenerife	Mujeres	3865.00
...	...	...	...	...
Población	2023	Vilaflor	Total	1823.00
Población. Proporción	2023	Vilaflor	Total	0.08
Población. Tasa de variación anual	2023	Vilaflor	Total	3.17
Población. Variación anual	2023	Vilaflor	Total	56.00

Tabla 2.3: Dataframe `df_poblacion_por_municipios_y_años`

2.3. Datos de turismo

A diferencia de los conjuntos de datos anteriores, la obtención de información no fue tan directa, ya que no se encontró un dataset que cumpliera con los requisitos que se tenía pensados para el estudio. En su lugar se trató de confeccionarlo, para el cual se partió del dataframe denominado [Pernoctaciones, viajeros alojados y entrados y estancia media según principales nacionalidades. Islas y municipios de Canarias por periodos](#) [8] del ISTAC [9].

Este dataset incluía una gran cantidad de variables, muchas de las cuales no resultaban relevantes para nuestro análisis, por lo que se descartaron hasta obtener una estructura como la mostrada en la tabla 2.4.

medidas#es	territorio#es	time_period#es	nacionalidad#es	obs_value
Pernoctaciones	Canarias	01/2009	Reino Unido	1903457.0
Pernoctaciones	Canarias	02/2009	Reino Unido	1717660.0
Viajeros alojados	El Pinar de El Hierro	04/2025	Dinamarca	3.0
...	...	...	...	...

Tabla 2.4: Dataframe df_turismo

A continuación, se filtraron los municipios exclusivos pertenecientes a la isla de Tenerife. Luego, se procedió a trabajar con la columna `medidas#es`, que distingue los [siguientes indicadores](#) [10]:

- *Pernoctaciones*: número total de plazas ocupadas cada uno de los días que componen el periodo de referencia, donde se entiende por plaza ocupada a la noche que un viajero se aloja en un establecimiento.
- *Viajeros entrados*: número de personas que realizan una o más pernoctaciones seguidas en el mismo alojamiento en el periodo de referencia.
- *Estancia media*: número de días que, por término medio, permanece un viajero entrado en un establecimiento en un periodo de referencia.
- *Viajeros alojados*: número de personas que realizan una o más pernoctaciones seguidas en el mismo alojamiento en el periodo de referencia, es decir, el número de viajeros entrados en el periodo de referencia más los viajeros que permanecen del periodo anterior.

De entre estas medidas, se seleccionó únicamente *Viajeros entrados*, ya que servirá como base para calcular, de manera aproximada, el número total de turistas en aquellos municipios para los que no se dispone de datos directos.

Posteriormente, se filtró la columna `nacionalidadES#es`, quedándonos únicamente con el valor *'Total'*, el cual representa la suma total de los viajeros sin segmentarlo por nacionalidades.

En cuanto a la columna `time_period#es` definimos una función con la cual añadimos una nueva columna con el año extraído y, además, limitamos el periodo de tiempo entre los años 2010 y 2023.

Finalizado este tratamiento, se observaron registros únicamente para los siguientes municipios: *'Adeje'*, *'Arona'*, *'Granadilla De Abona'*, *'Guia De Isora'*, *'La Laguna'*, *'Puerto De La Cruz'*, *'San Miguel'*,

'Santa Cruz De Tenerife', 'Santiago Del Teide' y 'Resto de Tenerife'. Este último recoge el número de turistas que se alojaron en el resto de municipios de la isla sin distinguir entre ellos.

Para imputar el número de turistas al resto de municipios, se aplicó una extrapolación proporcional en función de su población, utilizando la siguiente fórmula:

$$\text{turista_municipio} = \left(\frac{\text{df_turismo}['\text{Resto de Tenerife}']}{\text{diferencia_pobl_turis}} \right) \times \text{poblacion_municipio}$$

siendo

$$\text{diferencia_pobl_turis} = \text{poblacion_total_tenerife} - \left(\sum \text{poblacion_municipios_disponibles} \right)$$

El código utilizado para llevar a cabo este cálculo se encuentra descrito en el Apartado A.4 del anexo.

Tras este procesamiento, se obtuvo el dataframe `df_turismo_completo`, cuya estructura se muestra en la tabla 2.5. Las variables incluidas son:

- *medidas#es*: indica si el valor corresponde a un dato original del dataset ('Viajeros entrados') o si fue calculado mediante la extrapolación descrita ('Turismo Estimado').
- *territorio#es*: especifica el municipio al que pertenece la observación.
- *time_period#es*: mes y año de referencia del dato, en formato MM/AAAA.
- *nacionalidad#es*: indica si el dato es 'Total' (original) o 'Estimado' (dato calculado).
- *año*: año extraído de *time_period#es*.
- *mes*: mes extraído de *time_period#es*.

medidas#es	territorio#es	time_period#es	nacionalidad#es	obs_value	año	mes
Viajeros entrados	Adeje	01/2010	Total	125632.0	2010	NaN
Viajeros entrados	Adeje	02/2010	Total	113055.0	2010	NaN
...	...	...	...	...	...	...
Turismo Estimado	Vilaflor	11/2023	Estimado	17.0	2023	11.0
Turismo Estimado	Vilaflor	12/2023	Estimado	16.0	2023	12.0

Tabla 2.5: `df_turismo_completo`

2.4. Datos de empresas por municipios y actividad económica

Para complementar el análisis con información del tejido productivo de los municipios, se elaboró un dataframe específico denominado `df_empresas_por_municipios_y_act_economica`, que recoge el número de empresas por municipios y actividad económica. Los datos fueron obtenidos del Instituto Nacional de Estadística (INE), seleccionando el dataset [Empresas por municipio y actividad principal](#) [11], y aplicando un filtro para incluir exclusivamente los municipios pertenecientes a la isla de Tenerife.

El proceso de recogida y preparación de los datos se desarrolló en varias fases. Tras la descarga del archivo original desde la web del INE, se convirtió a formato CSV para facilitar su posterior tratamiento. A continuación, se cargó en el dataframe `df_empresas_por_municipios_y_act_economica`. Como parte de estandarización de datos, se adecuaron los nombres de los municipios para mejorar su legibilidad y asegurar la coherencia con los utilizados en otros conjuntos de datos empleados a lo largo del estudio.

La estructura del dataframe resultante puede consultarse en la tabla 2.6:

Totales Territoriales	Provincias	Municipios	Grupos CNAE	Periodo	Total
Total Nacional	38 Santa Cruz de Tenerife	38001 Adeje	Total	2023	4.900
Total Nacional	38 Santa Cruz de Tenerife	38001 Adeje	Total	2022	5.292
Total Nacional	38 Santa Cruz de Tenerife	38001 Adeje	Total	2021	5.014
...	...	...	...	...	...
Total Nacional	38 Santa Cruz de Tenerife	38052 Vilaflor de Chasna	Total servicios	2013	22
Total Nacional	38 Santa Cruz de Tenerife	38052 Vilaflor de Chasna	Total servicios	2012	22

Tabla 2.6: `df_empresas_por_municipios_y_act_economica`

A continuación, se describen las variables contenidas en este dataframe:

- *Totales Territoriales*: indica el nivel geográfico al que pertenece la observación.
- *Provincias*: provincia a la que pertenece el municipio registrado.
- *Municipios*: nombre del municipio al que se refiere la medida.
- *Grupos CNAE* [12]: código correspondiente a la Clasificación Nacional de Actividades Económicas, que identifica la actividad principal de las empresas.
- *Periodo*: año en el que se registró el dato.
- *Total*: valor de la observación.

Capítulo 3

Regresión lineal

3.1. Fundamentos teóricos

3.1.1. ¿Qué es la regresión lineal?

Para comprender el origen del análisis de regresión, es necesario remontarse al siglo XIX, cuando Francis Galton desarrolló los primeros estudios en este ámbito. Al investigar la relación entre la altura de padres e hijos, Galton observó que las alturas de los hijos tendían a 'regresar' hacia la media del grupo poblacional. A partir de esta observación, formuló una descripción matemática de dicha tendencia, lo que dio lugar a los primeros modelos de regresión.

El término regresión se conserva hasta hoy para describir las relaciones estadísticas entre variables. En el libro *Applied Linear Statistical Models* de Michael H. Kutner, Christopher J. Nachtsheim, John Neter y William Li [13], se define un modelo de regresión como un medio formal para expresar los dos ingredientes esenciales de una relación estadística:

- Una tendencia de la variable de respuesta Y a variar con la variable predictora X de manera sistemática.
- Una dispersión de puntos alrededor de la curva de la relación estadística.

Esta relación puede expresarse mediante la siguiente función lineal general:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \epsilon \quad (3.1)$$

donde:

- β_0 es el término independiente o intercepto.
- $\beta_1, \dots, \beta_k$ son los coeficientes de regresión que indican la influencia de cada predictor sobre la variable dependiente.
- ϵ representa el término de error, que recoge la variabilidad no explicada por el modelo.

3.1.2. Regresión lineal simple vs múltiple

Una vez definida la regresión lineal, es importante distinguir entre sus dos principales variantes: regresión lineal simple y regresión lineal múltiple.

En la regresión lineal simple, se analiza la relación entre la variable dependiente Y y una única variable independiente X . El modelo se representa mediante la siguiente expresión [14]:

$$Y = \beta_0 + \beta_1 X + \epsilon \quad (3.2)$$

donde:

- β_0 es el intercepto, es decir, valor que toma Y cuando $X = 0$.
- β_1 es la pendiente de la recta, es decir, indica cómo cambia Y al incrementar X en una unidad.
- ϵ es el término error, que agrupa los factores no observados que afecten a Y .

Tanto X como Y son variables aleatorias, por lo que no se puede establecer una relación lineal exacta entre ellas.

Por otro lado, la regresión lineal múltiple [15] generaliza el modelo simple al incorporar más de una variable explicativa. En este caso, se asume que una variable dependiente Y está influida por un conjunto de k predictores $x_1, \dots, x_k$. El modelo se expresa como:

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + \epsilon \quad (3.3)$$

donde:

- β_0 es el término independiente.
- $\beta_1, \dots, \beta_k$ son los coeficientes de regresión asociados a cada variable explicativa.
- ϵ representa el término de error, que recoge la influencia de todas aquellas variables no incluidas explícitas en el modelo.

Ambos modelos comparten la misma base conceptual, pero la regresión múltiple permite capturar relaciones más complejas al considerar múltiples factores que influyen simultáneamente sobre la variable de interés.

3.1.3. Hipótesis de partida

Para que los resultados de un modelo de regresión lineal sean válidos y fiables, se asume que cumplen una serie de requisitos, también conocidos como supuestos.

A continuación, se detallan los principales supuestos del modelo de regresión lineal [16, 17]:

- *Linealidad*: asume que existe una relación lineal entre la variable dependiente y las variables independientes. Esto significa que el efecto de cada predictor sobre la variable respuesta es aditivo y constante. Si este supuesto no se cumple, el modelo puede no captar correctamente la estructura de los datos, produciendo estimaciones poco representativas.

- *Normalidad*: se establece que los errores de la regresión deben seguir una distribución normal.
- *Homocedasticidad*: espera que la varianza de los errores sea constante para todos los valores de las variables independientes. En caso del incumplimiento de este supuesto, estaríamos hablando de heterocedasticidad, lo que significa que la dispersión de los errores varía en diferentes rangos de las variables predictoras.
- *Independencia*: los errores de la regresión no deben estar correlacionados entre sí.
- *No multicolinealidad* (únicamente en regresión múltiple): en los modelos de regresión lineal múltiple, se requiere que las variables explicativas no estén perfectamente correlacionadas entre sí. Si existe colinealidad alta, podría dificultar la estimación precisa de los coeficientes y reducir la estabilidad del modelo.

3.1.4. Métricas de evaluación

Cuando hablamos de métricas de evaluación en machine learning, nos referimos al conjunto de herramientas que permiten medir el rendimiento de un modelo y estimar su capacidad de generalización sobre datos futuros.

Entre algunas de las métricas más usadas podemos ver las siguientes [18-21]:

- *Error absoluto medio (MAE)*: halla la diferencia entre el valor pronosticado y el valor real en cada punto predecido:

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.4)$$

donde $|y_i - \hat{y}_i|$ representa el error absoluto para cada observación.

Si usamos esta medida nos encontraremos ante una métrica fácil de calcular donde, al usar valores absolutos, todos los errores se ponderan sobre la misma escala. Además, esta medida no penaliza los errores elevados, por lo tanto es útil si los datos de entrenamiento contiene valores atípicos.

A pesar de las ventajas mencionadas anteriormente, también tiene sus desventajas, entre las que se encuentran el hecho de no poder usarla para comparar series usando diferentes medidas ya que el *MAE* sigue una medida de precisión dependiente de la escala en la utiliza la misma escala que los datos que se están midiendo.

- *Error cuadrático medio (MSE)*: calcula el error elevando al cuadrado la diferencia entre el valor predicho y el valor real, y promediando el conjunto de datos. Resultando en la siguiente fórmula:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (3.5)$$

donde:

• y_i representa el valor real de la observación i .

- ` $\hat{y}_i$ indica el valor predicho por el modelo para la observación i .
- ` n es el número total de observaciones.

Entre algunas de las ventajas de usar *MSE* podemos destacar la penalización del modelo por el hecho de tener grandes errores al elevarlos al cuadrado ó cuando se trata de modelos lineales, la función de pérdida del *MSE* es convexa, lo que implica que no existen mínimos locales y garantiza una convergencia eficiente al mínimo global. Por otro lado, también tiene algunas desventajas como el hecho de ser muy sensible a los valores atípicos ya que al elevar los errores al cuadrado, los grandes errores toman un peso desproporcionado.

La diferencia principal entre el *MAE* y el *MSE* es que el *MAE* nos proporciona la diferencia absoluta, mientras que el *MSE* nos da la diferencia al cuadrado.

- *Raíz del error cuadrático medio (RMSE)*: es la raíz cuadrada del *MSE*. Con esta medida debemos considerar que si el valor del *RMSE* da como resultado 0, esto indica que el modelo tiene un ajuste perfecto y, por lado, tener en cuenta que cuanto menor sea el *RMSE*, mejor será el modelo y sus predicciones.

Su fórmula es la siguiente:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (3.6)$$

donde:

- ` y_i representa el valor real de la observación i .
- ` $\hat{y}_i$ indica el valor predicho por el modelo para la observación i
- ` n es el número total de observaciones

Entre sus ventajas nos encontramos con su facilidad de entendimiento y, por otro lado, una de sus principales desventajas es que, en comparación con el *MAE*, no es tan resistente a los valores atípicos.

- *Coeficiente de determinación (R^2)*: conocida también como prueba de bondad de ajuste, se trata de una medida que indica el rendimiento del modelo y nos indica que tan bien se ajusta el modelo a las observaciones reales, sin embargo, no indica si es bueno o malo, si los datos están sesgados o se eligió el método correcto.

Con esta medida obtendremos un valor que varía entre 0 y 1, el cual nos indicará un ajuste débil para valores bajos y, viceversa, un ajuste alto para valores cercanos a 1. Donde, entendamos el ajuste como la indicación para decirnos si sus valores predichos se desvían mucho de sus datos reales. Se puede interpretar dicho ajuste de la siguiente forma:

- ` $R^2 < 0,5$: ajuste débil.
- ` $0,5 \leq R^2 \leq 0,8$: ajuste moderado.
- ` $R^2 > 0,8$: ajuste fuerte.

Su fórmula se expresa de la siguiente manera:

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (3.7)$$

donde

- ` y_i : son los valores de la variable independiente.
- ` $\hat{y}_i$: representa los valores predichos del modelo.
- ` $\bar{y}$: es la media de y .

La principal ventaja que se puede encontrar al usar esta medida es el hecho de indicarnos qué tan bueno es el ajuste del modelo. En cambio, una de sus principales desventajas es la pérdida de significado de la medida R^2 para los modelos no lineales.

3.2. Regresión lineal simple

En este apartado nos centraremos en la construcción y evaluación del modelo aplicado en este estudio. Para ello, se presentará un ejemplo, partiendo del dataframe que se muestra parcialmente en la tabla 3.1:

municipio	año	total_turistas	total_residuos	total_poblacion	total_empresas
Adeje	2012	3230156.0	43162.920015	46894	3.843
Adeje	2013	3280748.0	43624.030010	49387.0	3.774
Adeje	2014	3414322.0	45073.730019	46667.0	3.903
...	...	...	...	...	...
Vilaflor	2021	105.0	872.059999	1789.0	108.000
Vilaflor	2022	162.0	867.079996	1767.0	111.000
Vilaflor	2023	181.0	847.839999	1823.0	105.000

Tabla 3.1: Dataframe regresión lineal simple

Para este ejemplo, nos planteamos la pregunta: ¿la cantidad de población influye en la generación de residuos?. Para responderla, se define como variable dependiente (Y) el campo `total_residuos`, y como variable independiente (X) el campo `total_poblacion`. Una vez definidas las variables, se construye y ajusta el modelo, mostrando a continuación algunos datos claves y la gráfica correspondiente.

Para ello, se ejecutó el código que se muestra a continuación (Código 3.1):

```

1 # Seleccionamos las variables
2 variable_x = 'total_poblacion'
3 variable_y = 'total_residuos'
4
5 # Creamos el modelo utilizando la fórmula y los datos
6 modelo = smf.ols(formula=f'{variable_y} ~ {variable_x}',
7                  data=df_combinado_regresion_lineal_1)
8
9 # Ajustamos el modelo a los datos
10 resultado = modelo.fit()
11
12 # 'resultado' contiene la información del modelo ajustado
13 print(resultado.summary())
14
15 print("Ecuación de la recta:", f"y = {round(resultado.params[0], 3)} +
16     {round(resultado.params[1], 3)}*x")
17
18 print("Coeficiente de correlación:",
19       round(df_combinado_regresion_lineal_1[[variable_x,
20       variable_y]].corr().iloc[0, 1], 3))
21
22 print("Coeficiente de determinación:", round(resultado.rsquared, 3))
23
24 sn.regplot(x=df_combinado_regresion_lineal_1[variable_x],
25            y=df_combinado_regresion_lineal_1[variable_y], color='red',
26            scatter_kws={'color': 'green'})
27
28 plt.xlabel("Población")
29 plt.ylabel("Residuos")
30 plt.show()

```

Listing 3.1: Código regresión lineal simple ejemplo 1

La ejecución del código anterior produce el siguiente resultado por pantalla de la Figura 3.1

Ecuación de la recta: $y = -2012.553 + 0.629 * x$
Coeficiente de correlación: 0.96
Coeficiente de determinación: 0.922

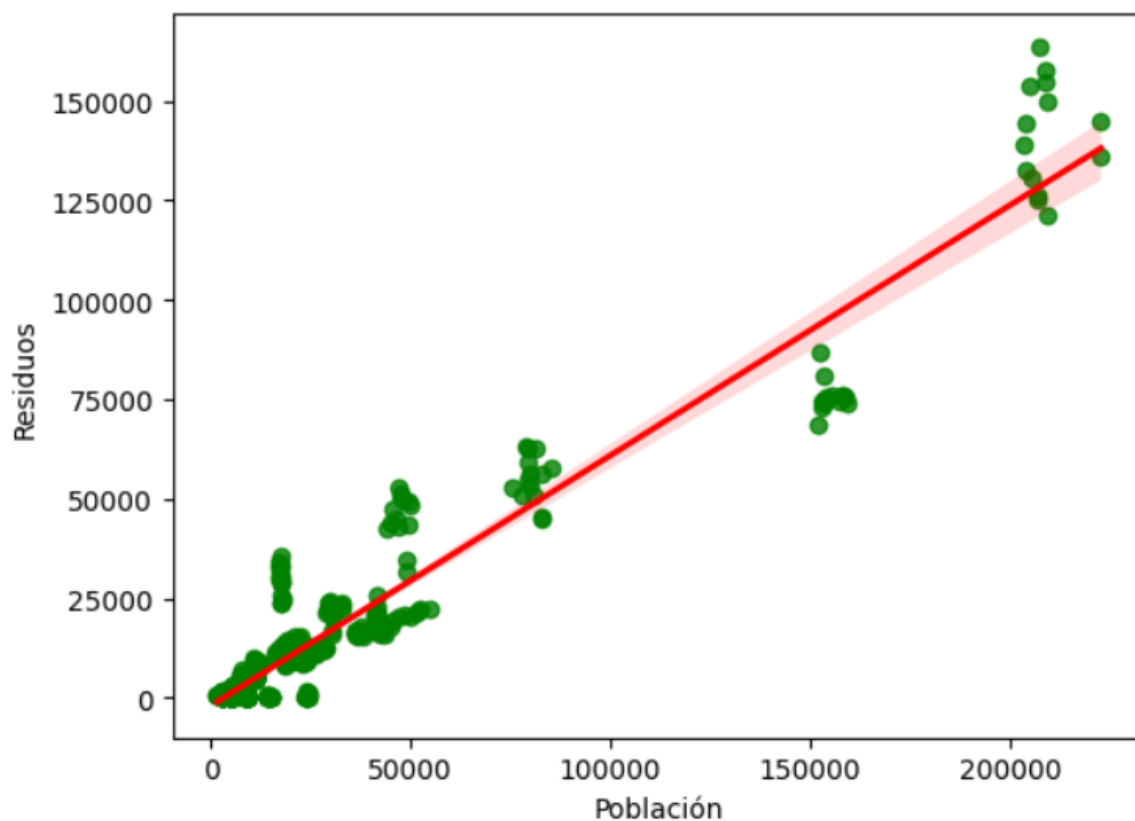


Figura 3.1: Resultado ejecución código 3.1

Como se puede observar, el modelo ofrece muy buenos resultados, reflejados en un coeficiente de correlación del 0,96 y un coeficiente de determinación del 0,92. A pesar de dicho resultados, el análisis mostrado sobre el modelo OLS (Figura 3.2) nos hace sospechar que el resultado no es de fiar.

OLS Regression Results						
Dep. Variable:	total_residuos	R-squared:	0.922			
Model:	OLS	Adj. R-squared:	0.921			
Method:	Least Squares	F-statistic:	5063.			
Date:	Sun, 22 Jun 2025	Prob (F-statistic):	2.39e-240			
Time:	19:41:08	Log-Likelihood:	-4511.4			
No. Observations:	433	AIC:	9027.			
Df Residuals:	431	BIC:	9035.			
Df Model:	1					
Covariance Type:	nonrobust					
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-2012.5530	468.933	-4.292	0.000	-2934.233	-1090.873
total_poblacion	0.6294	0.009	71.158	0.000	0.612	0.647
Omnibus:	81.601	Durbin-Watson:	0.223			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	251.868			
Skew:	0.860	Prob(JB):	2.03e-55			
Kurtosis:	6.316	Cond. No.	6.37e+04			

Figura 3.2: Resultados modelo OLS

A continuación definiremos e interpretaremos los resultados con la ayuda del artículo sobre la [interpretación de los resultados del resumen de OLS \[22\]](#):

- Variable de dependencia: es la variable que se desea predecir, en este caso `total_residuos`.
- R-squared (0,922): indica que el 92.2 % de la variación en la cantidad de los residuos se explica por el modelo.
- Adj. R-squared (0,921): indica el número de predictores.
- F-statistic (5063): evalúa la importancia global del modelo de regresión. Un alto valor indica que, al menos una variable predictora en caso de contar con más de una, está significativamente relacionada con la cantidad de residuos generados. En este caso sólo contamos con una, así que se concluye que está fuertemente relacionada.
- Intercept (−2012,5530): sugiere que cuando todos los predictores sean nulos, de forma hipotética, la cantidad de residuos base es -2012.5530.
- `total_poblacion` (0,6294): indica que cada vez que la población aumente en una unidad, la cantidad de residuos aumentará también 0.6294.
- Omnibus (81,601): evalúa la normalidad. Un valor alto sugiere una desviación de la normalidad.
- Prob(Omnibus) (0,000): valor por encima del 0.05 indica que los valores que los residuos siguen una distribución normal.
- Skew (0,860): mide la asimetría de la distribución de los residuos. Un valor cercano a cero indica una distribución simétrica, mientras que valores negativos o positivos indican sesgo a la izquierda o derecha, respectivamente.

- Kurtosis (6,316): un valor de 3 indica una distribución normal, mientras que valores por encima o debajo indican 'colas' [23] mas pesadas o ligeras, respectivamente.
- Durbin-Watson (0,223): sirve para controlar la autocorrelación. Los valores cercanos a 2 indican no autocorrelación mientras que valores a 1 o mayores a 3 sugieren autocorrelación positiva o negativa, como se produce en esta ocasión.
- Prob(JB) ($2,03 * 10^{-55}$): evalúa la hipótesis nula. En este caso la rechaza ya que el valor no es superior a 0.05, lo cual indicaría que se acepta la hipótesis nula.
- Cond. No. ($6,37 * 10^4$): evalúa la multicolinealidad, donde valores superiores a 30 indican problemas de multicolinealidad.

Una vez explicados algunos de los resultados del modelo OLS se puede ver que a pesar de que los coeficientes de correlación y determinación indican una fuerte relación entre las variables, el modelo no cumple varios de los supuestos explicados en el apartado 3.1.3, como la normalidad de los residuos, la homocedasticidad o la independencia de los errores y los cuales demostraremos a continuación.

Para el estudio de normalidad [24] contamos con el Shaphiro-Wilk test y D'Agostino's K-squared test, los cuales son de los test de hipótesis más usados y en ambos se considera como hipótesis nula que los datos procedan de una distribución normal. En este caso se descarta usar el Shaphiro-Wilk test ya que se desaconseja para más de 50 datos por su elevada sensibilidad a pequeñas desviaciones. Por ello, usamos el D'Agostino's K-squared test de la siguiente forma:

```
1 # Residuos del modelo
2 residuos_modelo = resultado.resid
3
4 # Test de normalidad de 'D'Agostino-Pearson
5 stat, dagostino_test_p_valor = normaltest(residuos_modelo)
```

Listing 3.2: D'Agostino's K-squared test en regresión lineal simple

Si mostramos por pantalla la salida de dicho test (Código 3.2) nos mostrará un $p\text{-valor} \approx 1,91 * 10^{-18}$ el cual, al tener un valor inferior a 0.05, rechaza la hipótesis nula de normalidad y se concluye que los residuos no siguen una distribución normal.

A continuación, comprobaremos si cumple el supuesto sobre la homocedasticidad para el cual usaremos la prueba de Breusch-Pagan (Código 3.3) [25] donde su hipótesis nula es que la homocedasticidad está presente y, en caso contrario significaría que existe heterocedasticidad en su lugar.

```
1 # Test de Breusch-Pagan
2 bp_test = het_breuschpagan(residuos_modelo, resultado.model.exog)
3 etiquetas = ['Lagrange multiplier stadicistic', 'p-value', 'f-value', 'f p-value']
4 print(dict(zip(etiquetas, bp_test)))
```

Listing 3.3: Breusch-Pagan test en regresión lineal simple

La ejecución del código 3.3 darán los siguientes resultados:

```
'Lagrange multiplier stadicistic' : np.float64(128,88970207420272),
'p - value' : np.float64(7,169374035760352e - 30),
'f - value' : np.float64(182,66879475267191),
'fp - value' : np.float64(5,960946335513637e - 35).
```

Donde destacaremos el valor $p - valor \approx 7,17 * 10^{-30}$ (muy inferior a 0,05), por lo que se rechaza la hipótesis nula y se concluye que no se cumple la homocedasticidad.

Por último, comprobaremos el último supuesto que no se cumple, la independencia [26]. En este caso usaremos el test de Durbin-Watson para determinar si los residuos son independientes y no existe correlación. Para esta prueba se usará como hipótesis nula la no existencia entre los residuos y, como hipótesis alternativa, la presencia de correlación. En este punto no haría falta realizar el test posteriormente, ya que está disponible de manera explícita en el resumen de los resultados (Figura 3.2). Aunque si se desea ejecutar esta prueba más tarde, podemos usar la línea de código 3.4 donde el resultado que mostrará será 0,22290706746001618.

Para que esta prueba se considerase exitosa debería de haber salido un valor entre 1,5 – 2,5, lo cual no ha sido el caso y se ha hallado un valor inferior a dicho rango e indicaría un problema en la correlación.

```
1 # Test de Durbin-Watson
2 durbin_watson_test = durbin_watson(residuos_modelo)
```

Listing 3.4: Durbin-Watson test en regresión lineal simple

A pesar de los altos valores del coeficiente de determinación ($R^2 = 0,922$) y del coeficiente de correlación ($r \approx 0,96$), los diagnósticos del modelo indican que no se cumplen varios de los supuestos fundamentales del modelo de regresión lineal. En concreto:

- Los residuos no siguen una distribución normal, según el test de D'Agostino's K-squared ($p < 0,05$).
- Existe heterocedasticidad, según el test de Breusch-Pagan ($p < 0,05$).
- Se detecta autocorrelación positiva de los errores, según el estadístico de Durbin-Watson ($valor \approx 0,22$, inferior al umbral de 1.5-2.5).

Por lo tanto, aunque existe una aparente relación lineal fuerte entre la población y la generación de residuos, al no cumplirse las hipótesis requeridas por el modelo lineal simple, este modelo debe usarse con cautela.

3.3. Regresión lineal múltiple

En el apartado 3.2 hemos visto un ejemplo de regresión lineal simple y hemos intentado crear un modelo de predicción en el que únicamente se necesitaba una variable predictora pero, ¿y si necesitáramos más?. Entonces estaríamos hablando de modelos de regresión lineal múltiple, de los cuales podemos encontrar su definición en el apartado 3.1.2 y, a continuación intentaremos mostrar un ejemplo de cómo se aplicaría.

Comenzaremos creando el modelo, para el cual usaremos el mismo dataframe mostrado en la tabla 3.1 el cual es el mismo que definimos para la regresión lineal simple con la sutil diferencia de que en este caso el dataframe usado, pese a ser el mismo, es una copia y recibe el nombre de `df_modelo_multiple`, y ejecutaremos el código 3.5:

```

1 # Construimos el modelo de regresión lineal múltiple
2 formula_multiple = 'total_residuos ~ total_turistas + total_poblacion'
3
4 # Creamos el modelo
5 modelo_multiple = smf.ols(formula=formula_multiple, data=df_modelo_multiple)
6
7 # Ajustamos el modelo
8 resultado_multiple = modelo_multiple.fit()
9
10 # Imprimimos el resumen de los resultados
11 print(resultado_multiple.summary())

```

Listing 3.5: Código para la regresión lineal múltiple

Una vez creado el modelo, imprimimos los resultados del modelo OLS (Figura 3.3) donde, a pesar de obtener un buen coeficiente de determinación (0,938), verificaremos a continuación el cumplimiento de los supuestos.

OLS Regression Results						
=====						
Dep. Variable:	total_residuos	R-squared:	0.938			
Model:	OLS	Adj. R-squared:	0.938			
Method:	Least Squares	F-statistic:	2787.			
Date:	Thu, 26 Jun 2025	Prob (F-statistic):	5.08e-223			
Time:	15:57:16	Log-Likelihood:	-3821.0			
No. Observations:	371	AIC:	7648.			
Df Residuals:	368	BIC:	7660.			
Df Model:	2					
Covariance Type:	nonrobust					
=====						
	coef	std err	t	P> t	[0.025	0.975]

Intercept	-2987.9774	458.071	-6.523	0.000	-3888.742	-2087.213
total_turistas	0.0060	0.001	11.112	0.000	0.005	0.007
total_poblacion	0.6057	0.009	68.664	0.000	0.588	0.623
=====						
Omnibus:	145.929	Durbin-Watson:	0.239			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	879.104			
Skew:	1.540	Prob(JB):	1.27e-191			
Kurtosis:	9.883	Cond. No.	9.41e+05			
=====						

Figura 3.3: Resultados OLS modelo múltiple

Para comprobar los supuestos se ejecutará el código disponible en el Anexo B.2 del cual extraemos las conclusiones mostradas en la tabla 3.2.

Supuesto	Test	Resultado	¿Se cumple?
Normalidad	D'Agostino-Pearson	$p \approx 2,05 * 10^{-32}$	No
Linealidad	RESET Test	$p \approx 7,46 * 10^{-61}$	No
Homocedasticidad	Breusch-Pagan	$p \approx 1,88 * 10^{-28}$	No
Independencia de errores	Durbin-Watson	$DW = 0,239$	No
Multicolinealidad	VIF	$VIF \approx 1,18$	Sí

Tabla 3.2: Resultados tests de supuestos en regresión lineal múltiple

Como podemos observar, la normalidad, linealidad y homocedasticidad no se cumple debido a que sus valores p – *valor* son inferiores a 0,05, en cambio, la independencia de errores falla ya que el test Durbin-Watson debería tener un valor dentro del rango 1,5 – 2,5.

Para intentar solucionar esto, aplicamos la transformación de las variables a logarítmicas (Anexo B.3) y, aunque parece mejorar algunos de los valores (Tabla 3.3), sigue sin ser suficiente e incluso convierte a test negativo la multicolinealidad.

Supuesto	Test	Resultado	¿Se cumple?
Normalidad	D'Agostino-Pearson	$p \approx 2,98 * 10^{-7}$	No
Linealidad	RESET Test	$p \approx 6,20 * 10^{-8}$	No
Homocedasticidad	Breusch-Pagan	$p \approx 9,40 * 10^{-31}$	No
Independencia de errores	Durbin-Watson	$DW = 0,230$	No
Multicolinealidad	VIF	$VIF \approx 14,10$	No

Tabla 3.3: Resultados tests de supuestos en regresión lineal múltiple después de la transformación

Al no cumplirse los supuestos, pese a intentar transformar el modelo a logarítmico, no se puede afirmar que se trate de un buen modelo.

Capítulo 4

Árboles de regresión

4.1. Fundamentos teóricos

4.1.1. Definición

El origen de los árboles de decisión se remonta al año 1984, con la publicación del libro *Classification and Regression Trees (CART)*, elaborado por L. Breiman, J. Friedman, R. Olshen y C. Stone [27].

Un árbol de decisión es un modelo predictivo que organiza decisiones y sus posibles consecuencias mediante una estructura jerárquica en forma de árbol. Dependiendo del tipo de la variable objetivo, se pueden distinguir dos variantes principales: árboles de clasificación, cuando la variable a predecir es cualitativa, y árboles de regresión, cuando dicha variable es cuantitativa.

En este caso, hablamos sobre árboles de regresión, dado que el objetivo es predecir una variable numérica. En cambio, si la variable dependiente fuera categórica, se aplicaría un árbol de clasificación [28].

Estos tipos de modelos se representan, visualmente como estructuras tipo árbol, similares a la que se muestra en la figura 4.1:

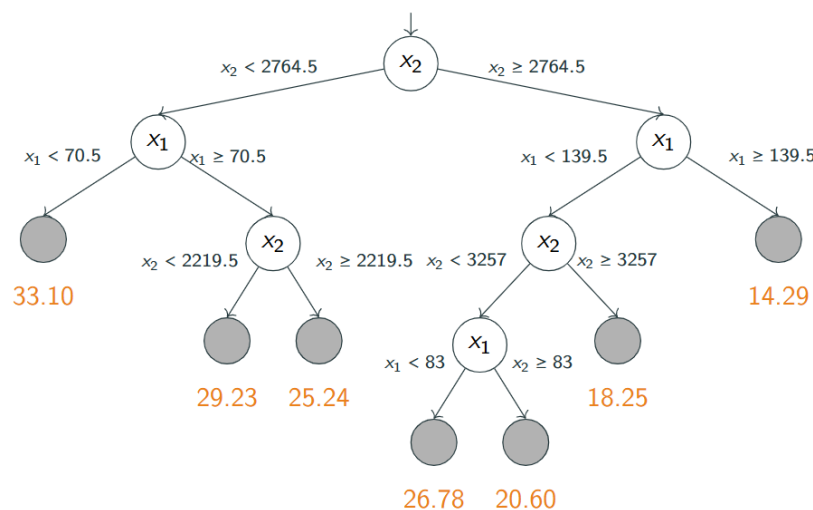


Figura 4.1: Estructura de un árbol de regresión

Los nodos interiores de un árbol de regresión se corresponden con todas o algunas de las variables predictoras del modelo. Las ramas que parten de estos nodos se asocian con los valores posibles

de estas variables (si estas son discretas) o con condiciones sobre el valor de las mismas (si son continuas). Los nodos finales del árbol representan la predicción que se hace de la variable a predecir. Así, por ejemplo, se tiene que si $x_2 < 2764,5$ y $x_1 < 70,5$ el valor predicho de la variable dependiente es 33,10.

4.1.2. Descripción del algoritmo de aprendizaje

Se pueden encontrar diferentes definiciones de cómo funciona el algoritmo de aprendizaje, aunque en la web dedicada a la [ciencia de datos \[29\]](#) se explica de la siguiente manera.

El entrenamiento de un árbol de predicción se divide en dos fases:

- División sucesiva del espacio de los predictores generando regiones no solapantes $R_1, R_2, \dots, R_j$. Aunque teóricamente las regiones pueden tener cualquier forma, si se limitan las regiones a forma rectangular, de esta manera se simplifica el proceso de construcción e interpretación.
- Predicción de la variable respuesta de cada región.

Aunque hemos visto que el proceso de construcción del árbol es bastante sencillo, la cuestión es establecer la metodología que permita decidir las divisiones de los predictores y sus valores o, lo que es lo mismo, crear las regiones. Es en este punto donde se diferencian los árboles de regresión y clasificación.

Nos centraremos en los árboles de regresión. En los cuales el criterio empleado con más frecuencia para identificar las divisiones es el RSS. El objetivo es encontrar las J regiones minimizando el RSS total:

$$RSS = \sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2 \quad (4.1)$$

donde $\hat{y}_{R_j}$ es la medida de la variable respuesta en la región R_j . Esto es equivalente a decir que se busca una distribución de regiones tal que, el sumatorio de las desviaciones al cuadrado entre las observaciones y la media de la región a la que pertenecen sea lo menor posible.

Como no es posible considerar todas las particiones posibles, se recurre a la división binaria recursiva. La cual sigue la misma idea que la selección de predictores en la regresión lineal múltiple.

En la división binaria recursiva, el objetivo es encontrar, en cada iteración, el predictor X_j y el punto de corte (umbral) s tal que, si se distribuyen las observaciones en las regiones $X|X_j < s$ y $X|X_j \geq s$ se consiga la mayor reducción posible en el RSS.

El algoritmo completo sería:

1. El proceso se inicia en lo más alto del árbol, donde todas las observaciones pertenecen a la misma región.
2. Se identifican los posibles puntos de corte s para cada uno de los predictores. Como en este caso hablamos de predictores continuos, se ordenarían de menor a mayor sus valores y el punto intermedio entre cada par de valores se usa como punto de corte.

3. Calcula el RSS total que se consigue con cada posible división identificada en el paso 2.

$$\sum_{i: x_i \in R_1(j,s)} (y_i - \hat{y}_{i_{R_1}})^2 + \sum_{i: x_i \in R_2(j,s)} (y_i - \hat{y}_{i_{R_2}})^2 \quad (4.2)$$

donde el primer término es el RSS de la primera región y segundo término pertenece al RSS de la segunda región.

4. Se selecciona el predictor X_j y el punto de corte s que resulta en el menor RSS total. En caso de que coincidan dos o más divisiones, la elección es aleatoria.
5. Se repiten del paso 1 al 4 para cada región que se ha creado en la iteración anterior hasta que se llega a algún criterio de parada como pueden ser haber establecido la profundidad máxima, alcanzar un máximo de nodos terminales, que no se reduzca el error en al menos un %, entre otras.

4.2. Implementación del modelo

En este apartado se describe el proceso de construcción para los árboles de regresión, detallando las variables utilizadas como características (predictoras), la variable objetivo (respuesta), y el procedimiento de preparación y partición del conjunto de datos. Además, se especifican las librerías usadas y los parámetros utilizados durante la implementación del modelo.

El desarrollo del modelo se llevó a cabo utilizando el lenguaje de programación Python, con el apoyo de librerías como:

- pandas y numpy para la gestión y manipulación de los datos.
- scikit-learn para la construcción del árbol de regresión y la evaluación de su rendimiento.
- matplotlib y seaborn para la visualización de resultados.

El conjunto de datos se preparó seleccionando las variables relevantes y aplicando transformaciones necesarias para su correcta interpretación por el modelo. Posteriormente, se dividió el dataset en dos subconjuntos:

- Conjunto de entrenamiento: utilizado para ajustar los parámetros del modelo.
- Conjunto de prueba: reservado para evaluar el rendimiento del modelo con datos no vistos durante el entrenamiento.

Esta división se realizó usando la función `train_test_split` de `scikit-learn`, empleando un 80 % del conjunto para entrenamiento y un 20 % para prueba.

A continuación, se presentan las tres versiones del modelo desarrolladas, las cuales difieren en ciertos aspectos como la selección del dataset o las variables predictoras usadas.

4.2.1. Versión 1

En esta primera versión del modelo, no se limita el análisis a un único municipio, sino que se pretende predecir el peso de los vertidos a nivel global para toda la isla de Tenerife.

El primer paso consiste en la preparación del conjunto de datos, representado por el dataframe `df_arbol_v1`, el cual se construye integrando diversos conjuntos de datos previamente tratados. En particular, se parte de tres dataframes individuales:

- `df_ar_tur`, derivado del conjunto final de datos turísticos, al que se le aplica el tratamiento descrito en la Apartado C.1.
- el dataframe de datos poblacionales, `df_ar_pob`, preparado según lo indicado en la Apartado C.2.
- los datos de residuos, guardados en `df_ar_res` y tratados conforme a lo detallado en la Apartado C.3

Una vez procesados estos tres dataframes como deseamos, se combinan en un único dataframe siguiendo los pasos descritos en la Apartado C.4. El resultado es un dataset consolidado con las variables necesarias para la construcción del modelo. La estructura final del conjunto se muestra en la Tabla 4.1.

municipio_mes	año	mes	peso_vertido	total_turismo	total_poblacion
Adeje	2013	2	3228.930000	124248.0	49387.0
Adeje	2013	3	3556.990007	152204.0	49387.0
...	...	...	...	...	...
Vilaflor	2023	11	70.340000	17.0	1823.0
Vilaflor	2023	12	71.380000	16.0	1823.0

Tabla 4.1: `df_arbol_v1`

Con el dataset ya preparado, procedemos a definir las variables que participarán en el modelo:

- Variables predictoras: conjunto de características relacionadas con la población, turismo y mes.
- Variable objetivo: `peso_vertido`, es decir, la cantidad de residuos en toneladas.

El conjunto se divide en datos de entrenamiento (test) y datos de prueba (validación) siguiendo el procedimiento descrito en la Apartado C.5, empleando una proporción 80/20 y fijando una semilla.

Para comenzar, se entrena el modelo del árbol de regresión básico, sin configurar ningún hiperparámetro. Sin embargo, esta configuración inicial muestra un comportamiento problemático: el modelo genera 2870 hojas a partir de 2873 registros en el conjunto de entrenamiento. Este resultado sugiere que el árbol está creando prácticamente una hoja por muestra, lo que indica un claro caso de overfitting (sobreajuste). Ante este escenario, se opta por ajustar los hiperparámetros del modelo.

Uno de los primeros hiperparámetros en analizar es la profundidad máxima del árbol (`max_depth`). A través de la evaluación del rendimiento del modelo con distintos valores para este parámetro, se

genera una gráfica que indica que un rango adecuado para `max_depth` podría situarse entre 6 y 14 aproximadamente.

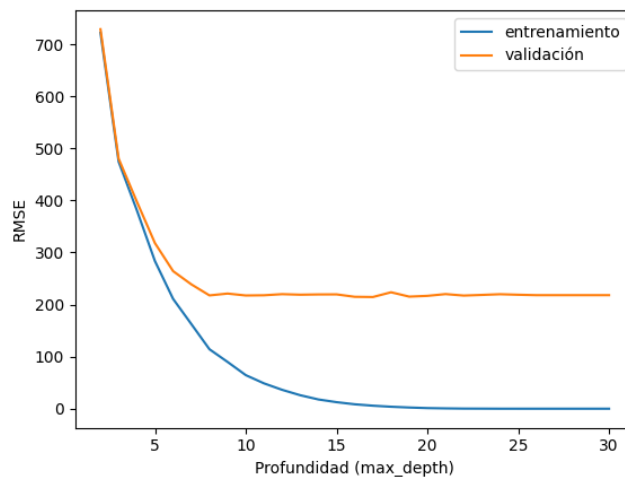


Figura 4.2: Estudio `max_depth`

A continuación, se analiza el efecto del parámetro `min_samples_leaf`, el cual indica el número mínimo de muestras necesarias para formar una hoja en el árbol. Este parámetro ayuda a controlar el crecimiento del árbol, evitando hojas con muy pocos datos que pueden llevar al sobreajuste. Tras ejecutar y visualizar la evolución del error para diferentes valores, se puede observar que los valores óptimos de `min_samples_leaf` se encuentran aproximadamente entre 6 y 10, donde se logra un buen equilibrio entre la precisión del modelo y su capacidad de generalización.

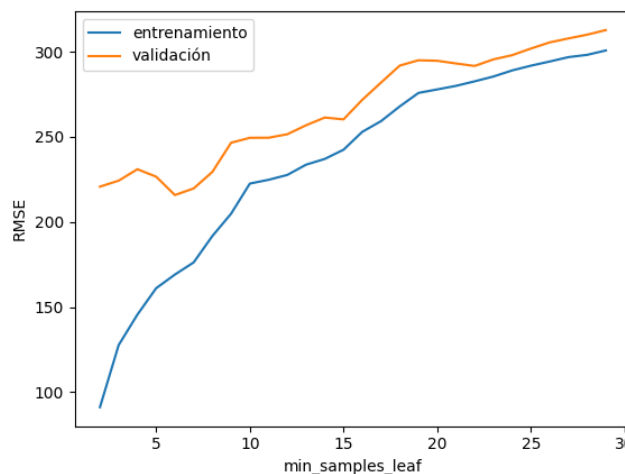


Figura 4.3: Estudio `min_samples_leaf`

Del mismo modo, se evalúa el parámetro `max_leaf_nodes`, que controla el número máximo de hojas que el árbol puede tener. Limitar este parámetro también contribuye a reducir la complejidad del modelo.

Según la gráfica obtenida, el rendimiento del modelo es mejor cuando `max_leaf_nodes` se encuentra en un rango aproximado entre 70 y 100, donde se observa una reducción del error sin provocar un sobreajuste evidente.

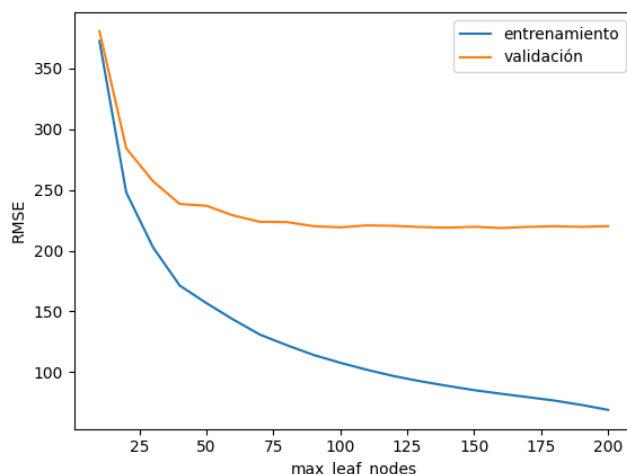


Figura 4.4: Estudio max_leaf_nodes

Una vez determinados los rangos óptimos para los hiperparámetros, se procede a la construcción del modelo definitivo utilizando GridSearchCV, con el objetivo de encontrar la mejor combinación posible de parámetros para el árbol de regresión, siguiendo los pasos mostrados en el Apartado C.6. Al ejecutar dicho código, se obtiene el árbol de decisión entrenado con los mejores hiperparámetros encontrados por GridSearchCV. Este árbol se muestra en la Figura 4.5, donde se puede observar la estructura generada y cómo se segmentan las decisiones en función de las variables predictoras.

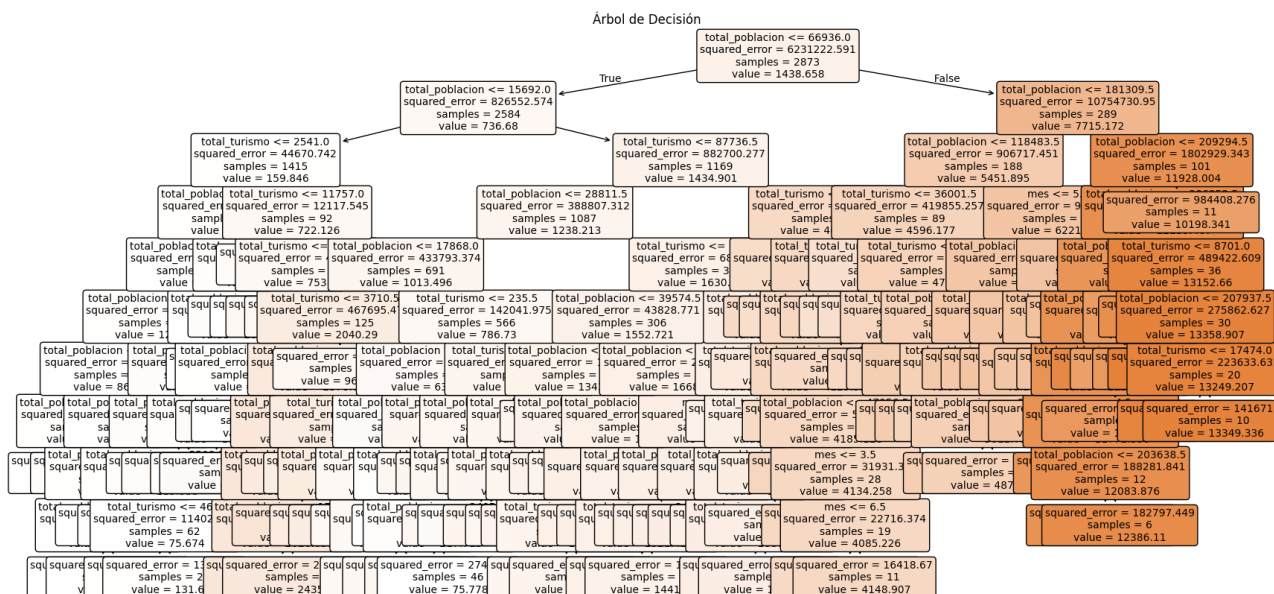


Figura 4.5: Árbol de regresión v1

El árbol generado con la mejor combinación de hiperparámetros encontrada ($\text{max_depth} = 10$, $\text{max_leaf_nodes} = 100$, $\text{min_samples_leaf} = 6$) muestra un rendimiento razonablemente satisfactorio. El error en el conjunto de prueba medido mediante la raíz del error cuadrático medio (RMSE) representa aproximadamente un 14.93 % del valor medio de la variable objetivo, mientras que el error absoluto medio (MAE) supone únicamente un 6.44 % del valor medio. Estos resultados

sugieren que, aunque existen márgenes de mejora, el modelo logra capturar una parte significativa del comportamiento de los datos sin caer en un sobreajuste evidente.

4.2.2. Versión 2

En esta segunda versión del modelo, se mantiene el plantamiento general descrito en la versión anterior, pero con una modificación clave: el análisis se limita exclusivamente al municipio del Puerto de la Cruz.

La elección de este municipio se debe a su relevancia como destino turístico y a su tamaño poblacional considerable, lo que lo convierte en un caso destacable.

El conjunto de datos se elabora a partir de los tres dataframes principales descritos en el capítulo 2:

- `df_residuos_total`
- `df_poblacion_por_municipios_y_años`
- `df_turismo`

A partir de ellos, se aplica un proceso de filtrado para aislar únicamente las observaciones correspondientes al municipio seleccionado. Una vez filtrados, los tres dataframes se combinan siguiendo el mismo procedimiento descrito para la versión 1, dando lugar, en este caso, al conjunto final de trabajo: `df_modelo_puerto_de_la_cruz`.

Este dataset (tabla 4.2) se encuentra organizado mensualmente dentro del periodo entre el año 2013 y 2023, permitiendo capturar la evolución temporal de las variables durante una década.

Cabe destacar que, como parte del preprocesamiento, se elimina la columna 'año' al no aportar valor predictivo adicional en este contexto.

mes	peso_vertido	total_turistas	total_poblacion
2.0	1702.279996	56573.0	28929.0
3.0	1863.750003	62616.0	28929.0
4.0	1717.790005	50687.0	28929.0
...	...	...	...
11.0	1779.449998	66055.0	30849.0
12.0	1857.340001	62539.0	30849.0

Tabla 4.2: `df_modelo_puerto_de_la_cruz`

Una vez preparado el conjunto de datos, se procede con la implementación del modelo de árbol de regresión, utilizando el mismo enfoque que en la versión anterior pero, en este caso, siguiendo los pasos de la Apartado C.7. el cual permite evaluar si un modelo centrado en un solo municipio mejora la capacidad predictiva en comparación con un modelo general con datos de toda la isla.

Una vez aplicamos el algoritmo, obtenemos como resultado el siguiente árbol (Figura 4.6):

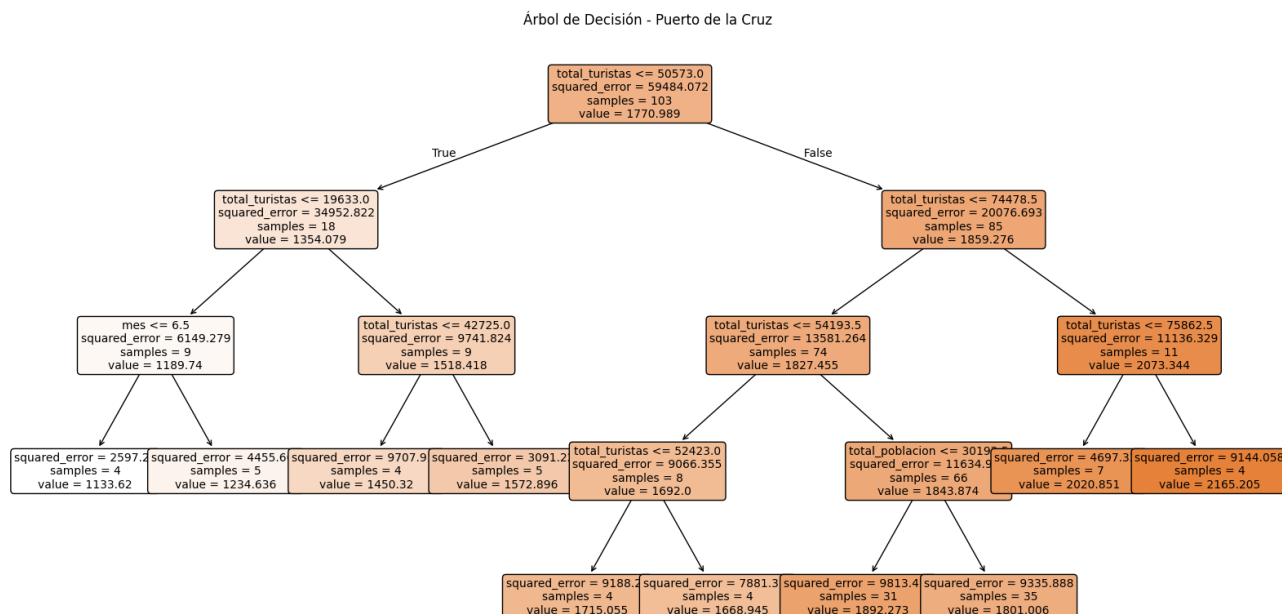


Figura 4.6: df_arbol_regresion_Puerto_de_la_Cruz

El modelo entrenado se evalúa sobre los conjuntos de entrenamiento y prueba, obteniendo los siguientes resultados:

- RMSE (entrenamiento) = 91.15
- RMSE (prueba) = 109.87
- MAE (entrenamiento) = 72.22
- MAE (prueba) = 91.82

Aunque estos valores puedan parecer elevados en términos absolutos, es fundamental contextualizarlos en función de la escala de la variable objetivo (peso_vertido). Para ello, se calcula qué porcentaje representan los errores (RMSE y MAE) respecto al valor medio de la variable objetivo en el conjunto de prueba.

El cálculo se puede realizar siguiendo las siguientes líneas de código:

```

1 error_pct_RMSE = rmse_test / y_test.mean()
2 print(f"RMSE del Puerto de la Cruz en test representa el {error_pct:.2%} del
   valor medio")
3
4 error_pct_mae = mae_test / y_test.mean()
5 print(f"MAE del Puerto de la Cruz representa el {error_pct_mae:.2%} del valor
   medio")
  
```

Listing 4.1: Porcentaje RMSE y MAE

Los resultados obtenidos son los siguientes:

- RMSE del Puerto de la Cruz en test representa el 6.26 % del valor medio.
- MAE del Puerto de la Cruz representa el 5.23 % del valor medio.

Estos valores indican que el modelo presentan un rendimiento notablemente satisfactorio. Los errores cometidos por el modelo son relativamente bajos en comparación con los valores reales de la variable objetivo, lo que sugiere que el modelo ha logrado capturar una parte importante del comportamiento real del peso de los vertidos en el municipio del Puerto de la Cruz.

4.2.3. Versión 3

Esta última versión replica la metodología aplicada en la versión anterior, con la diferencia principal de que no se limita el análisis a único municipio. En esta ocasión, se aplica el mismo procedimiento de construcción y entrenamiento del modelo para cada municipio de la isla de Tenerife, generando un modelo de regresión por separado para cada uno de ellos.

Dado que mostrar el código para cada árbol y representarlo gráficamente implicaría ocupar un número considerable de páginas, cuya única diferencia sería el nombre del municipio, se opta por mostrar los resultados en forma de tabla. Estos resultados se obtienen a partir de la adaptación del código del Anexo C.8 para cada municipio, ya que en este caso es para el municipio de Santa Cruz.

La tabla 4.3 recoge los errores absolutos, RMSE y MAE, obtenidos tras entrenar los modelos individuales por municipio.

Municipio	Prof.	Hojas	RMSE entrena.	RMSE test	MAE entrena.	MAE test
Adeje	5	14	194.27	297.75	152.86	228.49
Arafo	4	11	15.70	18.44	12.36	14.31
Arico	4	8	48.19	69.32	38.12	50.56
Arona	6	30	151.46	251.00	121.98	217.45
Buenavista del Norte	4	12	10.17	12.98	7.59	10.52
Candelaria	5	17	52.70	101.63	41.06	74.14
El Rosario	4	9	151.48	186.12	120.68	147.11
El Sauzal	2	4	3.01	2.05	2.29	1.82
El Tanque	3	5	2.88	4.19	1.43	2.50
Fasnia	4	8	14.38	14.11	10.62	11.83
Garachico	2	4	3.36	15.68	2.42	5.07
Granadilla de Abona	4	15	58.43	106.17	46.01	86.86
Guía de Isora	3	7	67.55	81.65	54.84	62.46
Güímar	5	17	34.24	44.87	26.56	39.00
Icod de los Vinos	4	13	36.81	47.42	29.95	34.49
La Guancha	2	4	1.78	2.35	1.33	1.77
La Matanza	2	4	1.58	1.55	1.14	1.10
La Orotava	2	4	85.78	73.18	68.83	54.23
La Victoria	8	30	6.05	14.64	4.45	9.66
Los Realejos	3	8	57.57	69.77	42.72	56.62
Los Silos	4	11	5.93	7.33	4.73	5.12
Puerto de la Cruz	4	10	91.15	109.87	72.22	91.82
La Laguna	4	14	189.80	208.94	147.29	161.08
San Juan de la Rambla	2	4	1.95	1.19	1.22	1.02
San Miguel	7	21	66.08	87.00	50.38	71.33
Santa Cruz de Tenerife	3	6	791.97	950.31	614.51	622.94
Santa Úrsula	4	10	8.80	11.47	6.90	8.56
Santiago del Teide	4	9	45.24	59.07	34.51	51.47
Tacoronte	8	30	9.18	13.92	5.96	11.73
Tegueste	6	11	26.24	49.92	20.41	38.23
Vilaflor	4	14	5.26	8.50	4.21	7.16

Tabla 4.3: Resultados del árbol de regresión por municipio

Sin embargo, los valores absolutos pueden dificultar la interpretación directa de los resultados, ya que no ofrecen una referencia clara sobre la magnitud del error en relación a los datos reales. Por ello, se construye una segunda tabla (tabla 4.4) en la que los errores se expresan como porcentaje del valor medio del peso_vertido en el conjunto de prueba de cada municipio. Esta representación

relativa permite una comparación más intuitiva entre municipios.

Municipio	% RMSE test	% MAE test	% RMSE entrena.	% MAE entrena.
Adeje	7.88 %	6.05 %	5.07 %	3.99 %
Arafo	7.90 %	6.13 %	6.89 %	5.43 %
Arico	14.48 %	10.56 %	10.03 %	7.94 %
Arona	5.45 %	4.73 %	3.25 %	2.62 %
Buenavista del Norte	7.15 %	5.80 %	5.66 %	4.22 %
Candelaria	9.68 %	7.06 %	5.08 %	3.95 %
El Rosario	7.76 %	6.14 %	6.31 %	5.02 %
El Sauzal	50.92 %	45.12 %	72.67 %	55.17 %
El Tanque	438.66 %	261.41 %	126.14 %	62.64 %
Fasnia	13.88 %	11.63 %	13.75 %	10.15 %
Garachico	262.16 %	84.77 %	87.30 %	62.98 %
Granadilla de Abona	6.36 %	5.21 %	3.48 %	2.74 %
Guía de Isora	8.40 %	6.42 %	6.97 %	5.66 %
Güímar	5.94 %	5.15 %	4.55 %	3.53 %
Icod de los Vinos	5.87 %	4.27 %	4.61 %	3.75 %
La Guancha	74.68 %	56.07 %	69.47 %	52.02 %
La Matanza	86.26 %	60.87 %	74.30 %	53.77 %
La Orotava	4.39 %	3.25 %	5.24 %	4.20 %
La Victoria	83.34 %	55.00 %	27.81 %	20.45 %
Los Realejos	5.10 %	4.14 %	4.24 %	3.15 %
Los Silos	4.37 %	3.05 %	3.60 %	2.87 %
Puerto de la Cruz	6.26 %	5.23 %	5.15 %	4.08 %
La Laguna	3.32 %	2.56 %	3.07 %	2.38 %
San Juan de la Rambla	81.38 %	69.73	113.62 %	71.08 %
San Miguel	7.83 %	6.42 %	5.82 %	4.43 %
Santa Cruz de Tenerife	8.07 %	5.29 %	6.61 %	5.13 %
Santa Úrsula	48.50 %	36.20 %	31.61 %	24.79 %
Santiago del Teide	8.38 %	7.31 %	6.28 %	4.79 %
Tacoronte	29.82 %	25.11 %	18.77 %	12.19 %
Tegueste	11.58 %	8.87 %	6.25 %	4.86 %
Vilaflor	12.27 %	10.34 %	7.63 %	6.10 %

Tabla 4.4: Porcentajes errores por municipio

Al observar los resultados de la tabla 4.4, podemos ver que aquellos municipios con peores resultados de precisión son:

- El Sauzal
- El Tanque
- Garachico
- La Guancha
- La Matanza
- La Victoria
- San Juan de la Rambla
- Santa Úrsula

El bajo rendimiento de estos modelos puede deberse a diversos factores, siendo uno de los más relevantes la escasez de datos disponibles en estos municipios. Esta menor disponibilidad de registros limita la capacidad del modelo para aprender patrones significativos, afectando negativamente a la calidad de la predicción.

Capítulo 5

Conclusiones

En este estudio se ha llevado a cabo un análisis sobre la generación de residuos en distintos municipios de Tenerife, utilizando modelos de regresión lineal, tanto simple como múltiple, y árboles de decisión, en concreto de regresión. El objetivo principal ha sido comprobar la viabilidad de predecir la cantidad de residuos generados a partir de variables independientes como la población o el número de turistas.

Los resultados obtenidos muestran que, si bien los modelos de regresión presentan coeficientes de determinación elevados lo cuál podría indicar que se trata de un buen modelo, su capacidad predictiva real no es adecuada según el estudio de los supuestos. Esto sugiere que las relaciones entre las variables no son estrictamente lineales o que existen otros factores que afectan significativamente a la generación de residuos.

Por otro lado, los árboles de regresión han ofrecido un mejor rendimiento general, con menores errores para la mayoría de los municipios. No obstante, en algunos casos concretos los errores fueron considerablemente más altos. Esta variabilidad puede estar relacionada con la escasa calidad o cantidad de datos disponibles en dichos municipios, lo que representa una de las principales limitaciones del estudio.

De cara a futuras líneas de trabajo, existen varias direcciones para mejorar y ampliar este estudio:

- Incorporación de nuevas variables explicativas, como la renta media por habitante, nivel sociocultural, modelos de gestión adoptados por las empresas en la gestión de sus materias, etc, que podrían aportar mayor capacidad explicativa al modelo.
- Exploración de modelos más avanzados, como Random Forest o redes neuronales, que podrían captar mejor las relaciones no lineales y reducir errores.
- Aplicación de técnicas de análisis temporal mediante series temporales como ARIMA ó redes neuronales LSTM [30], para estudiar la evolución de los residuos a lo largo del tiempo y mejorar predicciones futuras.
- Profundización en el tratamiento de datos de manera que se lograse una base de datos más homogénea y robusta.
- Ampliación del alcance geográfico del estudio, extendiéndolo al resto de municipios de las Islas Canarias para obtener una visión más completa.

En conclusión, aunque el estudio ha permitido extraer conclusiones relevantes sobre los factores que influyen en la generación de los residuos, también se ha puesto de manifiesto la necesidad de seguir mejorando los modelos y la calidad/cantidad de datos para obtener predicciones más precisas y útiles para la toma de decisiones.

Capítulo 6

Conclusions

This study has carried out an analysis of waste generation in different municipalities of Tenerife using both simple and multiple linear regression models, as well as decision trees, specifically regression trees. The main objective was to assess the feasibility of predicting the amount of waste generated based on independent variables such as population or the number of tourists.

The results obtained show that, although the regression models presented high coefficients of determination, which might indicate a good model, their actual predictive performance is not adequate according to the analysis of the assumptions. This suggests that the relationships between the variables are not strictly linear or that other significant factors influence waste generation.

On the other hand, regression trees demonstrated better overall performance, with lower errors for most municipalities. However, in certain specific cases, the errors were considerably higher. This variability may be related to the limited quality or quantity of available data in those municipalities, representing one of the main limitations of the study.

For future work, several directions can be considered to improve and expand this study:

- Incorporation of new explanatory variables, such as average income per capita, sociocultural level, waste management models adopted by companies, etc, which could enhance the model's explanatory power.
- Exploration of more advanced models, such as Random Forest or neural networks, which might better capture non-linear relationships and reduce errors.
- Application of time series analysis techniques such as ARIMA or LSTM neural networks, to study the evolution of waste over time and improve future predictions.
- Further development of data processing to achieve a more homogeneous and robust database.
- Expansion of the geographical scope of the study to include the rest of the municipalities in the Canary Islands in order to gain a more comprehensive view.

In conclusion, although the study has yielded relevant insights into the factors influencing waste generation, it has also highlighted the need to continue improving the models and the quality and quantity of data to obtain more accurate and useful predictions for decision-making.

Capítulo 7

Presupuesto

En este apartado se detallan los recursos humanos, técnicos y materiales utilizados para el desarrollo del presente Trabajo de Fin de Grado, con el objetivo de estimar un coste real en el supuesto de que se tratara de un contexto profesional y no académico.

Para presupuestar los recursos humanos, se ha considerado el salario bruto anual de un ingeniero de datos junior en España, estimado en 40000€, según datos de [Nuclio Digital School](#)[31] para profesionales con entre 1 y 3 años de experiencia. Esto equivale a un coste horario aproximado de 19,23€, asumiendo una jornada laboral estándar de 2080 horas anuales (40 horas semanales).

En cuanto a los recursos técnicos y materiales, el gasto es prácticamente nulo, ya que se han utilizado herramientas de software gratuitas. No obstante, se contemplan 500€ en conceptos de otros gastos, destinados a cubrir costes indirectos como el uso de ordenador, consumo eléctrico, entre otros.

Recurso	Tiempo estimado (h)	Coste total (€)
Análisis de datos	70	1346,10
Limpieza y tratamiento de datos	70	1346,10
Implementación de modelos	50	961,50
Otros gastos		500,00
Total	190	4153,70

Tabla 7.1: Presupuesto

Con base a la estimación anterior, el coste total del proyecto es de aproximadamente 4153,70€. Es importante destacar que una parte significativa de dicho presupuesto sería destinado a cubrir obligaciones fiscales, en caso de desarrollarse en un entorno profesional real.

Apéndice A

Algoritmos usados para el análisis y procesamiento de los datos

A.1. Función procesar_dataframe_residuos

```
1 /*****
2 *
3 * DESCRIPCION: Función usada para la creación de df\_residuos\_total
4 *
5 *****/
6
7
8 def procesar_dataframe_residuos(df, nombre_df):
9     print(f"Procesando {nombre_df}...")
10
11     # 1. Análisis Descriptivo (con estructura)
12     print(f"\nAnálisis Descriptivo para {nombre_df}:")
13
14     # Verificar si la columna 'i>¿municipio_nombre' existe y renombrarla
15     if 'i>¿municipio_nombre' in df.columns:
16         df.rename(columns={'i>¿municipio_nombre': 'municipio_nombre'},
17                     inplace=True)
18
19     print(f"Estructura de {nombre_df}:")
20     print(df.info()) # Muestra información general del DataFrame
21     print("\nPrimeras filas:")
22     print(df.head(4)) # Muestra las primeras 4 filas
23     print("\n-----\n")
24
25     print(df.describe()) # Estadísticas descriptivas básicas
26
27     # 2. Generación del reporte
28     print(f"\nGenerando reporte para {nombre_df}...")
29     report = ProfileReport(df, title=f'Residuos TF {nombre_df.split("_")[-1]}')
30     # Asume que nombre_df es 'df_residuosXX'
31     report.to_notebook_iframe()
32
33     # 3. Crear columna 'año'
34     año = int(nombre_df.split('residuos')[1]) # Extrae el año del nombre del
35         dataframe
36     df['año'] = año
37     print(f"\nColumna 'año' creada con valor {año}.")
```

```

37 # 4. Eliminar columna 'tercero_codigo_postal'
38 if 'tercero_codigo_postal' in df.columns:
39     df.drop(columns=['tercero_codigo_postal'], inplace=True)
40     print("\nColumna 'tercero_codigo_postal' eliminada.")
41 else:
42     print("\nAdvertencia: la columna 'tercero_codigo_postal' no existe en
        el DataFrame.")
43
44 # 5. Ajustar tipos de variables
45 df = ajustar_tipos_variables(df)
46 print(f"\nTipos de variables ajustados en {nombre_df}.")
47
48 # 6. Separación de mancomunidades
49 registros_mancomunidades =
        df[df['municipio_nombre'].str.contains('MANCOMUNIDAD')]
50 df = df[~df['municipio_nombre'].str.contains('MANCOMUNIDAD')]
51 print("\nFilas de mancomunidades separadas y eliminadas del DataFrame
        original.")
52
53 # 7. Modificar columna 'municipio_nombre' usando loc
54 df.loc[:, 'municipio_nombre'] =
        modificar_municipio_nombre(df['municipio_nombre'])
55 print("\nColumna 'municipio_nombre' modificada usando loc.")
56
57 # 8. Guardar el dataframe modificado
58 df.to_csv(f"{nombre_df}_limpio.csv", index=False)
59 print(f"\nDataframe {nombre_df} guardado como {nombre_df}_limpio.csv")
60
61 return df

```

A.2. Función ajustar_tipos_variables

```

1
2 /*****
3 *
4 * DESCRIPCION: Función usada para el ajuste de las variables de los dataframes
        de residuos.
5 *
6 *****/
7
8 def ajustar_tipos_variables(df):
9     tipos_deseados = {
10         'municipio_nombre': 'str',
11         'planta_nombre': 'category',
12         'planta_vertedero': 'bool',
13         'tipo_vertido_id': 'int64',
14         'tipo_vertido_nombre': 'category',
15         'tercero_codigo_postal': 'int64',
16         'tercero_ayuntamiento': 'bool',
17         'tercero_particular': 'bool',
18         'fecha': 'datetime64[ns]',
19         'peso_vertido': 'float64'

```

```

20     }
21
22     for columna, tipo in tipos_deseados.items():
23         if columna in df.columns:
24             try:
25                 df[columna] = df[columna].astype(tipo)
26             except ValueError as e:
27                 print(f"Error al convertir '{columna}' a {tipo}: {e}")
28             except Exception as e:
29                 print(f"Error inesperado al convertir '{columna}' a {tipo}: {e}")
30         else:
31             print(f"Advertencia: la columna '{columna}' no existe en el DataFrame.")
32
33     return df

```

A.3. Función modificar_municipio_nombre

```

1
2 /*****
3 *
4 * DESCRIPCION: Función usada para la estandarización del nombre de los
5 * municipios.
6 *****/
7
8 def modificar_municipio_nombre(municipio_nombre_series):
9     # Reemplazo de determinados nombres de municipios
10     municipio_nombre_series = municipio_nombre_series.str.replace(
11         'LAGUNA,LA', 'LA LAGUNA', regex=False)
12     municipio_nombre_series = municipio_nombre_series.str.replace(
13         'ROSARIO,EL', 'EL ROSARIO', regex=False)
14     municipio_nombre_series = municipio_nombre_series.str.replace(
15         'VICTORIA,LA', 'LA VICTORIA', regex=False)
16     municipio_nombre_series = municipio_nombre_series.str.replace(
17         'SAN SEBASTIÁN DE LA GOMERA', 'SAN SEBASTIÁN DE LA GOMERA',
18         regex=False)
19     municipio_nombre_series = municipio_nombre_series.str.replace(
20         'GUANCHA,LA', 'LA GUANCHA', regex=False)
21     municipio_nombre_series = municipio_nombre_series.str.replace(
22         'GÃ\x9cÃ\x8dMAR', 'GÜIMAR', regex=False)
23     municipio_nombre_series = municipio_nombre_series.str.replace(
24         'MATANZA,LA', 'LA MATANZA', regex=False)
25     municipio_nombre_series = municipio_nombre_series.str.replace(
26         'OROTAVA,LA', 'LA OROTAVA', regex=False)
27     municipio_nombre_series = municipio_nombre_series.str.replace(
28         'REALEJOS,LOS', 'LOS REALEJOS', regex=False)
29     municipio_nombre_series = municipio_nombre_series.str.replace(
30         'SAUZAL,EL', 'EL SAUZAL', regex=False)
31     municipio_nombre_series = municipio_nombre_series.str.replace(
32         'SILOS,LOS', 'LOS SILOS', regex=False)

```

```

33 municipio_nombre_series = municipio_nombre_series.str.replace(
34     'TANQUE,EL', 'EL TANQUE', regex=False)
35
36 # Convertimos a mayúscula y minúsculas
37 municipio_nombre_series = municipio_nombre_series.str.title()
38
39 return municipio_nombre_series

```

A.4. Algoritmo df_turismo_completo

```

1
2 /*****
3  *
4  * DESCRIPCION: Pasos seguidos el cálculo de turistas
5  *
6  *****/
7
8 # 1. Obtener la suma de toda la poblacion de Tenerife por años
9 poblacion_total_tenerife_anual =
10     df_poblacion_por_municipios_y_años_filtrado.groupby(
11         'año')['valor'].sum().to_dict()
12 # 2. Obtener la suma de la población de los municipios específicos por año
13 municipios_seleccionados = [
14     'Adeje',
15     'Arona',
16     'Granadilla De Abona',
17     'Guia De Isora',
18     'La Laguna',
19     'Puerto De La Cruz',
20     'San Miguel',
21     'Santa Cruz De Tenerife',
22     'Santiago Del Teide'
23 ]
24 poblacion_municipios_seleccionados_anual =
25     df_poblacion_por_municipios_y_años_filtrado[
26         df_poblacion_por_municipios_y_años_filtrado['territorio'].isin(
27             municipios_seleccionados)
28     ].groupby('año')['valor'].sum().to_dict()
29
30 # 3. Calcular la diferencia de población por año
31 diferencia_poblacion_anual = {}
32
33 for año, poblacion_total in poblacion_total_tenerife_anual.items():
34     poblacion_seleccionados = poblacion_municipios_seleccionados_anual.get(año,
35         0)
36     diferencia_poblacion_anual[año] = poblacion_total - poblacion_seleccionados
37
38 # 4. Calcular la proporción de turismo por diferencia de población por mes y año
39 # Inicializamos un diccionario vacío para almacenar la proporción de turismo
40 # por diferencia de población para cada mes y año
41 proporcion_turismo_poblacion_mensual = {}

```



```

39
40 # Seleccionamos las filas del DataFrame df_turismo donde la columna
    'TERRITORIO#es' es 'Resto de Tenerife', establecemos la columna
    'TIME_PERIOD#es' como índice y luego iteramos a través de los pares
    índice-valor (mes/año - valor de turismo) de la columna 'OBS_VALUE'
41 for periodo_resto, turismo_resto in df_turismo[
42 df_turismo['TERRITORIO#es'] == 'Resto de
43 Tenerife'].set_index('TIME_PERIOD#es')['OBS_VALUE'].items():
44     # Utilizamos un bloque try-except para manejar posibles errores si el formato
        de la cadena en periodo_resto no es el esperado ('MM/AAAA') Hay años que
        son el total y se representan como AAAA.
45     try:
46         # Dividimos la cadena del periodo de tiempo para obtener el mes y el año
47         mes_resto, año_resto = periodo_resto.split('/')
48         # Convertimos el año a un entero para poder buscarlo en el diccionario
            diferencia_poblacion_anual
49         año_int = int(año_resto)
50         # Obtenemos la diferencia de población anual correspondiente al año
            actual desde el diccionario diferencia_poblacion_anual
51         # Usamos .get() para manejar el caso en que no haya datos de diferencia
            de población para ese año
52         diferencia_pobl_anual = diferencia_poblacion_anual.get(año_int)
53         # Verificamos que se haya encontrado la diferencia de población para el
            año y que sea un valor positivo para evitar divisiones por cero o
            valores no significativos
54         if diferencia_pobl_anual is not None and diferencia_pobl_anual > 0:
55             # Calculamos la proporción dividiendo el valor de turismo del
                'Resto de Tenerife' para el mes actual entre la diferencia de
                población anual de ese mismo año
56             proporcion = turismo_resto / diferencia_pobl_anual
57             # Almacenamos la proporción calculada en el diccionario
                proporcion_turismo_poblacion_mensual con el periodo de tiempo
                ('MM/AAAA') como clave
58             proporcion_turismo_poblacion_mensual[periodo_resto] = proporcion
59
60         # Se imprimen mensajes de advertencia si no se encuentra la diferencia
            de población para un año o si no es un valor positivo
61         elif diferencia_pobl_anual is None:
62             print(f"Advertencia: No se encontró la diferencia de
63                 población para el año {año_int} (mes: {periodo_resto}).")
64         elif diferencia_pobl_anual <= 0:
65             print(f"Advertencia: La diferencia de población no es positiva
66                 para el año {año_int} (mes: {periodo_resto}).")
67         except ValueError:
68             print(f"Formato de fecha no esperado en Resto de Tenerife:
                {periodo_resto}")
69
70 # 5. Calcular el turismo estimado por meses para los municipios faltantes
71 municipios_con_datos_turismo = df_turismo['TERRITORIO#es'].unique().tolist()
72 municipios_faltantes = [
73     mun
74     for mun in
75         df_poblacion_por_municipios_y_años_filtrado['territorio'].unique()
76     if mun not in municipios_con_datos_turismo and mun != 'Tenerife'
77         and mun != 'Resto de Tenerife'
78 ]

```

```

79 turismo_estimado_mensual = []
80
81 for municipio in municipios_faltantes:
82     poblacion_municipio_anual_df = df_poblacion_por_municipios_y_años_filtrado[
83         df_poblacion_por_municipios_y_años_filtrado['territorio'] == municipio
84     ].set_index('año')['valor'].to_dict()
85
86     for periodo_resto, proporcion in
87         proporcion_turismo_poblacion_mensual.items():
88             try:
89                 mes_resto, año_resto = periodo_resto.split('/')
90                 año_int = int(año_resto)
91                 poblacion_municipio = poblacion_municipio_anual_df.get(año_int)
92                 if poblacion_municipio is not None:
93                     turismo_estimado = proporcion * poblacion_municipio
94                     turismo_estimado_mensual.append(
95                         {
96                             'MEDIDAS#es': 'Turismo Estimado',
97                             'TERRITORIO#es': municipio,
98                             'TIME_PERIOD#es': periodo_resto,
99                             'NACIONALIDAD#es': 'Estimado',
100                             'OBS_VALUE': turismo_estimado,
101                             'año': año_int,
102                             'mes': int(mes_resto),
103                         }
104                     )
105             except ValueError:
106                 print(f"Error al procesar el periodo '{periodo_resto}'
107                     para el municipio '{municipio}'")
108
109 df_turismo_estimado_mensual = pd.DataFrame(turismo_estimado_mensual)
110
111 # Redondeamos OBS_VALUE para que no sea decimal
112 df_turismo_estimado_mensual['OBS_VALUE'] =
113     df_turismo_estimado_mensual['OBS_VALUE'].round()
114
115 # 6. Cocatenamos df_turismo y df_turismo_estimado para tener todos los datos de
116     turismo en una tabla
117 df_turismo_completo = pd.concat([df_turismo, df_turismo_estimado_mensual],
118     ignore_index=True)

```

Apéndice B

Algoritmos usados para la regresión lineal

B.1. Preparación df_ar_tur

```
1 /*****
2 *
3 *
4 * DESCRIPCION: Algoritmo usado para preparar el dataframe df_ar_tur.
5 *
6 *****/
7
8 # Preparamos los datos de turismo
9 df_ar_tur = df_turismo_completo.copy()
10
11 # Se esliminan las filas que contenga únicamente el año en TIME_PERIOD#es ya
12   que representa el total del año y aquí no nos interesa
13 df_ar_tur = df_ar_tur[df_ar_tur['TIME_PERIOD#es'].str.contains('/')]
14
15 # Convertimos la columna TIME_PERIOD#es a tipo fecha para poder manipularla
16   después
17 df_ar_tur['TIME_PERIOD#es'] = pd.to_datetime(df_ar_tur['TIME_PERIOD#es'],
18   format='%m/%Y')
19
20 # Necesitamos el mes, la columna mes es extraída de TIME_PERIOD#es para aquel
21   mes que está definida como NaN
22 df_ar_tur.loc[:, 'mes'] = df_ar_tur['TIME_PERIOD#es'].dt.month
23
24 # Eliminamos la columna TIME_PERIOD#es, MEDIDAS#es, NACIONALIDAD#es
25 df_ar_tur.drop(columns=['TIME_PERIOD#es', 'MEDIDAS#es', 'NACIONALIDAD#es'],
26   inplace=True)
27
28 # Renombramos OBS_VALUE a total_turismo
29 df_ar_tur.rename(columns={'OBS_VALUE': 'total_turismo'}, inplace=True)
30
31 # Convertimos el mes a int
32 df_ar_tur['mes'] = df_ar_tur['mes'].astype(int)
33
34 # Cambiamos el nombre de TERRITORIO#es a municipio_nombre
35 df_ar_tur.rename(columns={'TERRITORIO#es': 'municipio_nombre'}, inplace=True)
```

B.2. Evaluación de supuestos

```
1 /*****
2 *
3 * DESCRIPCION: Algoritmos usados para la evaluación de los supuestos en árboles
4   de regresión
5 *
6 *****/
7
8 # Multicolinealidad
9 # Extraemos las variables independientes
10 var_ind = df_modelo_multiple[['total_turistas', 'total_poblacion']]
11
12 # Calculamos el VIF
13 vif_data = pd.DataFrame()
14 vif_data["feature"] = var_ind.columns
15 vif_data["VIF"] = [variance_inflation_factor(var_ind.values, i)
16                   for i in range(len(var_ind.columns))]
17
18 # Imprimimos el VIF
19 print(vif_data)
20
21 # Homocedasticidad
22 # Test de Breusch-Pagan
23 bp_test = het_breuschpagan(resultado_multiple.resid,
24                             resultado_multiple.model.exog)
25 etiquetas = ['Lagrange multiplier stadistic', 'p-value', 'f-value', 'f p-value']
26 print(dict(zip(etiquetas, bp_test)))
27
28 # Normalidad
29 # Test de normalidad de 'DAgostino-Pearson
30 d_agostino = stats.normaltest(resultado_multiple.resid)
31 print("Test de normalidad de D'Agostino-Pearson:")
32 print("Estadístico de prueba:", d_agostino.statistic)
33 print("Valor p:", d_agostino.pvalue)
34
35 # Linealidad
36 # RESET Test
37 resultado_reset = linear_reset(resultado_multiple)
38 print("Resultado del RESET Test:")
39 print("Estadístico de prueba:", resultado_reset.statistic)
40 print("Valor p:", resultado_reset.pvalue)
```

B.3. Transformación a modelo logarítmico - logarítmico

```
1 /*****
2 *
3 *
4 * DESCRIPCION: Algoritmo usado para transformar a logarítmica las
5 * variables independientes y dependiente
6 *
7 *****/
8
9 df_modelo_multiple["log_total_residuos"] =
10     np.log(df_modelo_multiple["total_residuos"] + 1)
11
12 df_modelo_multiple["log_total_turistas"] =
13     np.log(df_modelo_multiple["total_turistas"] + 1)
14 df_modelo_multiple["log_total_poblacion"] =
15     np.log(df_modelo_multiple["total_poblacion"] + 1)
16
17 formula_log_log = 'log_total_residuos ~ log_total_turistas +
18     log_total_poblacion'
19
20 modelo_log_log = smf.ols(formula=formula_log_log, data=df_modelo_multiple)
21
22 resultado_log_log = modelo_log_log.fit()
23
24 print(resultado_log_log.summary())
```

Apéndice C

Algoritmos usados para los árboles de regresión

C.1. Preparación df_ar_tur

```
1 /*****
2 *
3 * DESCRIPCION: Algoritmo usado para preparar el dataframe df_ar_tur.
4 *
5 *****/
6
7 # Preparamos los datos de turismo
8 df_ar_tur = df_turismo_completo.copy()
9
10 # Se eliminan las filas que contenga únicamente el año en TIME_PERIOD#es ya
    que representa el total del año y aquí no nos interesa
11 df_ar_tur = df_ar_tur[df_ar_tur['TIME_PERIOD#es'].str.contains('/')]
12
13 # Convertimos la columna TIME_PERIOD#es a tipo fecha para poder manipularla
    después
14 df_ar_tur['TIME_PERIOD#es'] = pd.to_datetime(df_ar_tur['TIME_PERIOD#es'],
    format='%m/%Y')
15
16 # Necesitamos el mes, la columna mes es extraída de TIME_PERIOD#es para aquel
    mes que está definida como NaN
17 df_ar_tur.loc[:, 'mes'] = df_ar_tur['TIME_PERIOD#es'].dt.month
18
19 # Eliminamos la columna TIME_PERIOD#es, MEDIDAS#es, NACIONALIDAD#es
20 df_ar_tur.drop(columns=['TIME_PERIOD#es', 'MEDIDAS#es', 'NACIONALIDAD#es'],
    inplace=True)
21
22 # Renombramos OBS_VALUE a total_turismo
23 df_ar_tur.rename(columns={'OBS_VALUE': 'total_turismo'}, inplace=True)
24
25 # Convertimos el mes a int
26 df_ar_tur['mes'] = df_ar_tur['mes'].astype(int)
27
28 # Cambiamos el nombre de TERRITORIO#es a municipio_nombre
29 df_ar_tur.rename(columns={'TERRITORIO#es': 'municipio_nombre'}, inplace=True)
```

C.2. Preparación df_ar_pob

```
1 /*****
2  *
3  * DESCRPCION: Algoritmo usado para preparar el dataframe df_ar_pob.
4  *
5  *****/
6
7  # Preparamos los datos de población
8  df_ar_pob = df_poblacion_por_municipios_y_años.copy()
9
10 # Nos quedamos únicamente con tipo_medida = Población
11 df_ar_pob = df_ar_pob[df_ar_pob['tipo_medida'] == 'Población']
12
13 # Nos quedamos con filas de sexo = Total
14 df_ar_pob = df_ar_pob[df_ar_pob['sexo'] == 'Total']
15
16 # Eliminamos las filas donde territorio = 'Tenerife' ya que representan el
17   total de la isla
18 df_ar_pob = df_ar_pob[df_ar_pob['territorio'] != 'Tenerife']
19
20 # Cambiamos el nombre de valor a total_poblacion
21 df_ar_pob.rename(columns={'valor': 'total_poblacion'}, inplace=True)
22
23 # Eliminamos las columnas tipo_medida, sexo
24 df_ar_pob.drop(columns=['tipo_medida', 'sexo'], inplace=True)
25
26 # Cambiamos el nombre de territorio a municipio_nombre
27 df_ar_pob.rename(columns={'territorio': 'municipio_nombre'}, inplace=True)
```

C.3. Preparación df_ar_res

```
1 /*****
2  *
3  * DESCRPCION: Algoritmo usado para preparar el dataframe df_ar_res.
4  *
5  *****/
6
7 # Preparamos los datos de residuos
8 df_ar_res = df_residuos_total.copy()
9
10 # Eliminamos las filas NaN de fecha
11 df_ar_res.dropna(subset=['fecha'], inplace=True)
12
13 # Creamos una nueva nueva columna 'mes', donde extraemos el mes de fecha
14 df_ar_res.loc[:, 'mes'] = df_ar_res['fecha'].dt.month
15
16 # Eliminamos planta_nombre, planta_vertedero, tercero_ayuntamiento,
17     tercero_particular, fecha
18 df_ar_res.drop(columns=['planta_nombre', 'planta_vertedero',
19     'tercero_ayuntamiento', 'tercero_particular', 'fecha'], inplace=True)
20
21 # Agrupamos cada municipio por año y mes
22 df_ar_res = df_ar_res.groupby(['municipio_nombre', 'año', 'mes']).agg(
23     peso_vertido=('peso_vertido', 'sum')).reset_index()
24
25 # Eliminamos los municipio_nombre 'Sin Asignar', 'San Sebastián De La Gomera',
26     'Valle Gran Rey', 'Recogidas Agrupadas De Envases', 'Nan', 'El Hierro'
27 df_ar_res = df_ar_res[~df_ar_res['municipio_nombre'].isin(['Sin Asignar', 'San
28     Sebastián De La Gomera', 'Valle Gran Rey', 'Recogidas Agrupadas De Envases',
29     'Nan', 'El Hierro'])]
```


C.4. Creación df_arbol_v1

```
1 /*****
2 *
3 *
4 * DESCRPCION: Algoritmo usado para crear el dataframe final df_arbol_v1. Se
   usará para el modelo de árbol de regresión.
5 *
6 *****/
7
8 # Creamos el df_arbol_v1
9 # Comenzamos uniendo df_ar_res y df_ar_tur por municipio_nombre, año y mes.
10 df_arbol_v1 = pd.merge(
11     df_ar_res, df_ar_tur, on=['municipio_nombre', 'año', 'mes'], how='outer')
12
13 # Unimos a df_arbol_v1 el dataframe df_ar_pob por munnicipio_nombre y año
14 # (para todos los meses)
15 df_arbol_v1 = pd.merge(
16     df_arbol_v1, df_ar_pob, on=['municipio_nombre', 'año'], how='outer')
17
18 # Eliminamos las filas con peso_vertido NaN
19 df_arbol_v1.dropna(subset=['peso_vertido'], inplace=True)
```

C.5. Inicialización del modelo

```
1 /*****
2  *
3  * DESCRIPCION: Pasos para inicializar el modelo
4  *
5  *****/
6
7 # Antes debemos eliminar las columnas de municipio_nombre y año
8 df_arbol_v1.drop(columns=['municipio_nombre', 'año'], inplace=True)
9
10 # Arrays x y;
11 # Toma todas las filas y todas las columnas de las variables predictoras (es
12 # decir, todas las columnas excepto la última)
13 x = df_arbol_v1.iloc[:, :-1]
14 # Toma la columna de la variable a predecir, es decir, peso_vertido
15 y = df_arbol_v1.iloc[:, -1]
16
17 # Partición de los datos en sets de entrenamiento (x_train, y_train) y sets de
18 # prueba (x_test, y_test)
19 x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25,
20 # random_state=123)
```

C.6. Árbol de regresión v1

```
1
2
3 /*****
4  *
5  * DESCRIPCION: Algoritmo para la creación de la primera versión de árbol de
6  * regresión.
7  *
8  *****/
9
10 from sklearn.model_selection import GridSearchCV
11 from sklearn.tree import DecisionTreeRegressor, plot_tree
12 from sklearn.metrics import mean_squared_error
13 from sklearn.metrics import mean_absolute_error
14 from sklearn.ensemble import RandomForestRegressor
15 import matplotlib.pyplot as plt
16 import seaborn as sns
17 import pandas as pd
18 import numpy as np
19
20 # Definir variables predictoras (X) y variable objetivo (y)
21 X = df_arbol_v1[['total_turismo', 'total_poblacion', 'mes']]
22 y = df_arbol_v1['peso_vertido']
23
24 # Dividir datos en entrenamiento y validación en 75-25
```

```

24 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
    random_state=42)
25
26 # Definir el modelo base
27 arbol_v1 = DecisionTreeRegressor(random_state=42)
28
29 # Hiperparámetros a evaluar
30 param_grid = {
31     'max_depth': [6, 8, 9, 10, 11, 12, 13, 14],
32     'min_samples_leaf': [6, 8, 10, 12, 14],
33     'max_leaf_nodes': [70, 75, 80, 90, 95, 100]
34 }
35
36 # Búsqueda en rejilla con validación cruzada
37 grid_search = GridSearchCV(
38     estimator=arbol_v1,
39     param_grid=param_grid,
40     cv=5,
41     scoring='neg_root_mean_squared_error',
42     n_jobs=-1,
43     verbose=1,
44     return_train_score=True
45 )
46
47 # Ejecutar búsqueda
48 grid_search.fit(x_train, y_train)
49
50 # Resultados
51 print("Mejores hiperparámetros encontrados:")
52 print(grid_search.best_params_)
53
54 # Entrenar modelo final
55 mejor_modelo = grid_search.best_estimator_
56 mejor_modelo.fit(x_train, y_train)
57
58 # Evaluación
59 y_train_pred = mejor_modelo.predict(x_train)
60 y_test_pred = mejor_modelo.predict(x_test)
61
62 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
63 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
64 mae_train = mean_absolute_error(y_train, y_train_pred)
65 mae_test = mean_absolute_error(y_test, y_test_pred)
66
67 print(f"RMSE entrenamiento: {rmse_train:.2f}")
68 print(f"RMSE validación (test): {rmse_test:.2f}")
69 print(f"MAE entrenamiento: {mae_train:.2f}")
70 print(f"MAE validación (test): {mae_test:.2f}")
71
72 # Visualizar árbol
73 plt.figure(figsize=(20, 10))
74 plot_tree(mejor_modelo, feature_names=X.columns, filled=True, rounded=True,
    fontsize=10)
75 plt.title("Árbol de Decisión")
76 plt.show()
77
78 # Información del modelo

```

```

79 print(f"Profundidad del árbol: {mejor_modelo.get_depth()}")
80 print(f"Número de hojas del árbol: {mejor_modelo.get_n_leaves()}")
81 print(f"Parámetros del árbol: {mejor_modelo.get_params()}")
82
83 error_pct_RMSE = rmse_test / y_test.mean()
84 print(f"RMSE en test representa el {error_pct_RMSE:.2%} del valor medio")
85
86 error_pct_mae = mae_test / y_test.mean()
87 print(f"MAE representa el {error_pct_mae:.2%} del valor medio")

```

C.7. Árbol de regresión v2

```

1
2 /*****
3 *
4 * DESCRIPCION: Algoritmo para la creación de la segunda versión de árbol de
5   regresión.
6 *
7 *****/
8
9 # Eliminar filas con valores faltantes (NaN)
10 df_modelo_PC = df_modelo_puerto_de_la_cruz.dropna()
11
12 # Definir variables predictoras (X) y variable objetivo (y)
13 X = df_modelo_PC[['total_turistas', 'total_poblacion', 'mes']]
14 y = df_modelo_PC['peso_vertido']
15
16 # Dividir datos en entrenamiento y validación en 80-20
17 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
18   random_state=42)
19
20 # Definir el modelo base
21 arbol_v2_puerto_cruz = DecisionTreeRegressor(random_state=42)
22
23 # Hiperparámetros a evaluar
24 param_grid = {
25     'max_depth': [2, 3, 4, 5, 6, 7, 8, 10, 12],
26     'min_samples_leaf': [2, 4, 6, 7, 9],
27     'max_leaf_nodes': [30, 40, 50, 60, 70]
28 }
29
30 # Búsqueda en rejilla con validación cruzada
31 grid_search = GridSearchCV(
32     estimator=arbol_v2_puerto_cruz,
33     param_grid=param_grid,
34     cv=5,
35     scoring='neg_root_mean_squared_error',
36     n_jobs=-1,
37     verbose=1,
38     return_train_score=True
39 )

```

```

39 # Ejecutar búsqueda
40 grid_search.fit(x_train, y_train)
41
42 # Resultados
43 print("Mejores hiperparámetros encontrados:")
44 print(grid_search.best_params_)
45
46 # Entrenar modelo final
47 mejor_modelo = grid_search.best_estimator_
48 mejor_modelo.fit(x_train, y_train)
49
50 # Evaluación
51 y_train_pred = mejor_modelo.predict(x_train)
52 y_test_pred = mejor_modelo.predict(x_test)
53
54 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
55 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
56 mae_train = mean_absolute_error(y_train, y_train_pred)
57 mae_test = mean_absolute_error(y_test, y_test_pred)
58
59 print(f"RMSE entrenamiento: {rmse_train:.2f}")
60 print(f"RMSE validación (test): {rmse_test:.2f}")
61 print(f"MAE entrenamiento: {mae_train:.2f}")
62 print(f"MAE validación (test): {mae_test:.2f}")
63
64 # Visualizar árbol
65 plt.figure(figsize=(20, 10))
66 plot_tree(mejor_modelo, feature_names=X.columns, filled=True, rounded=True,
67           fontsize=10)
68 plt.title("Árbol de Decisión - Puerto de la Cruz")
69 plt.show()
70
71 # Información del modelo
72 print(f"Profundidad del árbol: {mejor_modelo.get_depth()}")
73 print(f"Número de hojas del árbol: {mejor_modelo.get_n_leaves()}")
74 print(f"Parámetros del árbol: {mejor_modelo.get_params()}")

```

C.8. Árbol de regresión v3

```
1 /*****
2 *
3 * DESCRIPCION: Algoritmo para la creación de la tercera versión de árbol de
4   regresión.
5 *
6 *****/
7
8 df_residuos_total_santa_cruz = df_residuos_total.copy()
9
10 df_residuos_total_santa_cruz =
11     df_residuos_total_santa_cruz[df_residuos_total_santa_cruz['municipio_nombre']
12     == 'Santa Cruz De Tenerife']
13
14 # Extraemos el mes de la columna fecha
15 df_residuos_total_santa_cruz.loc[:, 'mes'] =
16     df_residuos_total_santa_cruz['fecha'].dt.month
17
18 # Agrupamos por mes y año
19 df_residuos_total_santa_cruz = df_residuos_total_santa_cruz.groupby(['mes',
20     'año']).agg(peso_vertido=('peso_vertido', 'sum')).reset_index()
21
22 # Ahora extraemos de df_poblacion_por_municipios_y_años el territorio = 'Santa
23   Cruz de Tenerife' para tener la población única de este municipio
24 df_poblacion_por_municipios_y_años_santa_cruz =
25     df_poblacion_por_municipios_y_años.copy()
26 df_poblacion_por_municipios_y_años_santa_cruz =
27     df_poblacion_por_municipios_y_años[
28     df_poblacion_por_municipios_y_años['territorio'] == 'Santa Cruz De Tenerife']
29 df_poblacion_por_municipios_y_años_santa_cruz
30
31 # Nos quedamos con sexo=Total y tipo_medida=Población
32 df_poblacion_por_municipios_y_años_santa_cruz =
33     df_poblacion_por_municipios_y_años_santa_cruz[(df_poblacion_por_municipios_y_años[
34     'sexo'] == 'Total') & (df_poblacion_por_municipios_y_años['tipo_medida'] ==
35     'Población')]
36
37 # Filtramos df_turismo para obtener los datos de turismo de Santa Cruz de
38   Tenerife
39 df_turismo_santa_cruz = df_turismo.copy()
40 df_turismo_santa_cruz =
41     df_turismo_santa_cruz[df_turismo_santa_cruz['TERRITORIO#es'] == 'Santa Cruz
42     De Tenerife']
43
44 # Eliminamos aquellas filas en las que TIME_PERIOD#es sólo contenga el año
45   porque es el total y no podemos extraer el mes
46 df_turismo_santa_cruz =
47     df_turismo_santa_cruz[df_turismo_santa_cruz['TIME_PERIOD#es'].str.contains('/')]
48
49 # Extraemos el mes
50 df_turismo_santa_cruz['TIME_PERIOD#es'] =
51     pd.to_datetime(df_turismo_santa_cruz['TIME_PERIOD#es'], format='%m/%Y')
52 df_turismo_santa_cruz.loc[:, 'mes'] =
53     df_turismo_santa_cruz['TIME_PERIOD#es'].dt.month
```

```

36
37 # Agrupamos df_turismo_santa_cruz por año y mes
38 df_turismo_santa_cruz = df_turismo_santa_cruz.groupby(['año',
39     'mes']).agg(total_turistas=('OBS_VALUE', 'sum')).reset_index()
40
41 # Unimos en df_modelo_santa_cruz por año y mes
42 df_modelo_santa_cruz = pd.merge(df_residuos_total_santa_cruz,
43     df_turismo_santa_cruz, on=['año', 'mes'], how='outer')
44
45 # Al df_modelo_santa_cruz le añadimos el total_población aunque en este caso no
46 # lo tenemos por meses así que ponemos que todo el año no varía la población
47 # por mes
48 df_modelo_santa_cruz = pd.merge(df_modelo_santa_cruz,
49     df_poblacion_por_municipios_y_años_santa_cruz[['año', 'valor']], on='año',
50     how='left')
51
52 # Renombramos valor con total_poblacion
53 df_modelo_santa_cruz.rename(columns={'valor': 'total_poblacion'}, inplace=True)
54
55 # Como tenemos valores NaN, los eliminamos
56 df_modelo_santa_cruz.dropna(inplace=True)
57
58 # Eliminamos la columna de año
59 df_modelo_santa_cruz.drop(columns='año', inplace=True)
60
61 # 1. Eliminar filas con valores faltantes (NaN)
62 df_modelo_SC = df_modelo_santa_cruz.dropna()
63
64 # 2. Definir variables predictoras (X) y variable objetivo (y)
65 X = df_modelo_SC[['total_turistas', 'total_poblacion', 'mes']]
66 y = df_modelo_SC['peso_vertido']
67
68 # 3. Dividir datos en entrenamiento y validación en 80-20
69 x_train, x_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
70     random_state=42)
71
72 # 4. Definir el modelo base
73 arbol_v2_santa_cruz = DecisionTreeRegressor(random_state=42)
74
75 # 5. Hiperparámetros a evaluar
76 param_grid = {
77     'max_depth': [2, 3, 4, 5, 6, 7, 8, 10, 12],
78     'min_samples_leaf': [2, 4, 6, 7, 9],
79     'max_leaf_nodes': [30, 40, 50, 60, 70]
80 }
81
82 # 6. Búsqueda en rejilla con validación cruzada
83 grid_search = GridSearchCV(
84     estimator=arbol_v2_santa_cruz,
85     param_grid=param_grid,
86     cv=5,
87     scoring='neg_root_mean_squared_error',
88     n_jobs=-1,
89     verbose=1,
90     return_train_score=True
91 )

```

```

86 # 7. Ejecutar búsqueda
87 grid_search.fit(x_train, y_train)
88
89 # 8. Resultados
90 print("Mejores hiperparámetros encontrados:")
91 print(grid_search.best_params_)
92
93 # 9. Entrenar modelo final
94 mejor_modelo = grid_search.best_estimator_ mejor_modelo.fit(x_train, y_train)
95
96 # 10. Evaluación
97 y_train_pred = mejor_modelo.predict(x_train)
98 y_test_pred = mejor_modelo.predict(x_test)
99
100 rmse_train = np.sqrt(mean_squared_error(y_train, y_train_pred))
101 rmse_test = np.sqrt(mean_squared_error(y_test, y_test_pred))
102 mae_train = mean_absolute_error(y_train, y_train_pred)
103 mae_test = mean_absolute_error(y_test, y_test_pred)
104
105 print(f"RMSE entrenamiento: {rmse_train:.2f}")
106 print(f"RMSE validación (test): {rmse_test:.2f}")
107 print(f"MAE entrenamiento: {mae_train:.2f}")
108 print(f"MAE validación (test): {mae_test:.2f}")
109
110 # 11. Visualizar árbol
111 plt.figure(figsize=(20, 10))
112 plot_tree(mejor_modelo, feature_names=X.columns, filled=True, rounded=True,
113           fontsize=10)
114 plt.title("Árbol de Decisión - Santa Cruz")
115 plt.show()
116
117 # 12. Información del modelo
118 print(f"Profundidad del árbol: {mejor_modelo.get_depth()}")
119 print(f"Número de hojas del árbol: {mejor_modelo.get_n_leaves()}")
120 print(f"Parámetros del árbol: {mejor_modelo.get_params()}")
121
122 # 13. Porcentaje RMSE
123 error_pct_RMSE = rmse_test / y_test.mean()
124 print(f"RMSE de Santa Cruz de Tenerife en test representa el
125       {error_pct_RMSE:.2%} del valor medio")
126
127 # 14. Porcentaje MAE
128 error_pct_mae = mae_test / y_test.mean()
129 print(f"MAE de Santa Cruz de Tenerife representa el {error_pct_mae:.2%} del
130       valor medio")

```


Bibliografía

- [1] La Sexta. *Multitudinarias manifestaciones en 113 ciudades europeas para protestar contra el cambio climático*. 2024. URL: https://www.lasexta.com/noticias/sociedad/manifestaciones-ocho-ciudades-espanolas-protestar-cambio-climatico_20240601665b1739e73ed6000148ccf6.html.
- [2] Iberdrola. *COP29: Cumbre del Clima 2024*. 2024. URL: <https://www.iberdrola.com/sostenibilidad/contra-cambio-climatico/cop29>.
- [3] María Álvarez Ruiz. *Los residuos y los ODS*. 2021. URL: <https://www.comunidadism.es/los-residuos-y-los-ods/>.
- [4] ONU. *Objetivo 11: Lograr que las ciudades sean más inclusivas, seguras, resilientes y sostenibles*. URL: <https://www.un.org/sustainabledevelopment/es/cities/>.
- [5] ONU. *Objetivo 12: Garantizar modalidades de consumo y producción sostenibles*. URL: <https://www.un.org/sustainabledevelopment/es/sustainable-consumption-production/>.
- [6] Cabildo de Tenerife. *Diccionario de datos sobre la recogida de residuos*. 2023. URL: <https://datos.tenerife.es/es/datos/tablero?resourceId=36e003e5-1ecb-47a7-98c6-1ed1c2da41b8>.
- [7] Instituto Canario de Estadística. *Población según sexos. Municipios por islas de Canarias y años*. 2023. URL: https://www3.gobiernodecanarias.org/istac/statistical-visualizer/visualizer/data.html?resourceType=dataset&agencyId=ISTAC&resourceId=E30245A_000002&version=1.2#visualization/table.
- [8] Instituto Canario de Estadística. *Pernoctaciones, viajeros alojados y entrados y estancia media según principales nacionalidades. Islas y municipios de Canarias por periodos*. 2025. URL: https://www3.gobiernodecanarias.org/istac/statistical-visualizer/visualizer/data.html?resourceType=dataset&agencyId=ISTAC&resourceId=C00065A_000036&version=~latest#visualization/table.
- [9] Instituto Canario de Estadística. *Instituto Canario de Estadística*. URL: <https://www.gobiernodecanarias.org/istac/>.
- [10] Instituto Canario de Estadística. *Infomación - Pernoctaciones, viajeros alojados y entrados y estancia media según principales nacionalidades. Islas y municipios de Canarias por periodos*. 2025. URL: https://www3.gobiernodecanarias.org/istac/statistical-visualizer/visualizer/data.html?resourceType=dataset&agencyId=ISTAC&resourceId=C00065A_000036&version=~latest#visualization/table.
- [11] Instituto Nacional de Estadística. *Empresas por municipio y actividad principal*. URL: https://www.ine.es/jaxiT3/Datos.htm?t=4721#_tabs-tabla.
- [12] Julián Vida Barea. *Clasificación Nacional de Actividades Económicas*. URL: <https://www.cnae.com.es/index.php>.

- [13] Michael H. Kutner et al. *Applied Linear Statistical Models*. 5.^a ed. New York: McGraw-Hill Irwin, 2005. ISBN: 0-07-238688-6.
- [14] M. Carmen Carollo Limeres. *Regresión Lineal Simple*. 2012. URL: http://eio.usc.es/eipc1/BASE/BASEMASTER/FORMULARIOS-PHP-DPTO/MATERIALES/Mat_50140116_Regr_%20simple_2011_12.pdf.
- [15] Ismael Sánchez Borrego. *Regresión Múltiple*. URL: <https://wpd.ugr.es/~bioestad/wp-content/uploads/M%C3%A9todos-de-regresi%C3%B3n-M%C3%A9tricas-de-regresión-Múltiple.pdf>.
- [16] A. Castro. *Regresión lineal: supuestos*. 2023. URL: https://rpubs.com/acastro/regresion_lineal_supuestos.
- [17] Gareth James et al. *An Introduction to Statistical Learning with Applications in R*. Springer, 2013. ISBN: 978-1-4614-7137-0.
- [18] Esteban Madrigal. *Conoce las métricas de precisión más comunes para Modelos de Regresión*. 2022. URL: <https://www.growupcr.com/post/metricas-precision>.
- [19] Nagesh Singh Chauhan. *Métricas De Evaluación De Modelos En El Aprendizaje Automático*. 2023. URL: <https://www.datasource.ai/es/data-science-articles/metricas-de-evaluacion-de-modelos-en-el-aprendizaje-automatico>.
- [20] 4Geeks. *Métricas de Evaluación*. URL: <https://4geeks.com/es/lesson/metricas-de-evaluacion>.
- [21] Kjell Johnson Max Kuhn. *Applied Predictive Modeling*. Springer, 2013. ISBN: 978-1-4614-6848-6.
- [22] GeeksforGeeks. *Interpreting the results of Linear Regression using OLS Summary*. 2024. URL: <https://www.geeksforgeeks.org/machine-learning/interpreting-the-results-of-linear-regression-using-ols-summary/>.
- [23] LibreTexts. *Áreas de Colas de Distribuciones*. URL: [https://espanol.libretexts.org/Esadisticas/Estadisticas_Introductorias/Libro%3A_Estad%C3%ADsticas_Introductorias_\(Shafer_y_Zhang\)/05%3A_Variables_aleatorias_continuas/5.04%3A_%C3%81reas_de_Colas_de_Distribuciones](https://espanol.libretexts.org/Esadisticas/Estadisticas_Introductorias/Libro%3A_Estad%C3%ADsticas_Introductorias_(Shafer_y_Zhang)/05%3A_Variables_aleatorias_continuas/5.04%3A_%C3%81reas_de_Colas_de_Distribuciones).
- [24] Joaquin Amat Rodrigo. *Análisis de normalidad con python*. 2021. URL: <https://cienciadedatos.net/documentos/pystats06-analisis-normalidad-python>.
- [25] GeeksforGeeks. *How to Perform a Breusch-Pagan Test in Python*. 2022. URL: <https://www.geeksforgeeks.org/python/how-to-perform-a-breusch-pagan-test-in-python/>.
- [26] Zach Bobbitt. *How to Perform a Durbin-Watson Test in Python*. 2024. URL: <https://www.statology.org/durbin-watson-test-python/>.
- [27] Leo Breiman et al. *Classification and Regression Trees*. 1.^a ed. Chapman y Hall/CRC, 1984. ISBN: 978-0412048418.
- [28] Freddy Hernández Barajas. *Árboles de regresión*. 2024. URL: https://fhernanb.github.io/libro_mod_pred/arb-de-regre.html.
- [29] Joaquín Amat Rodrigo. *Árboles de decisión con Python: regresión y clasificación*. 2020. URL: https://cienciadedatos.net/documentos/py07_arboles_decision_python.
- [30] Konstantin Kutzkov. *ARIMA vs Prophet vs LSTM for Time Series Prediction*. 2025. URL: <https://neptune.ai/blog/arima-vs-prophet-vs-lstm>.

- [31] Carlos Palmón. *¿Cuánto ganan los Data Scientist, Data Analyst e ingenieros de Datos en 2025? Salarios Data Science, Data Analytics y Data Engineer*. 2025. URL: <https://nuclio.school/blog/salario-data-scientist-data-analyst-ingeniero-de-datos-analista-de-big-data/>.